

```

# match features and estimate homography
matches = sift.match_twosided(d0,d1)
ndx = matches.nonzero()[0]
fp = homography.make_homog(l0[ndx,:2].T)
ndx2 = [int(matches[i]) for i in ndx]
tp = homography.make_homog(l1[ndx2,:2].T)

model = homography.RansacModel()
H = homography.H_from_ransac(fp,tp,model)

```

Now we have a homography that maps points on the marker (in this case the book) in one image to their corresponding locations in the other image. Let's define our 3D coordinate system so that the marker lies in the X-Y plane ($Z = 0$) with the origin somewhere on the marker.

To check our results, we will need some simple 3D object placed on the marker. Here we will use a cube and generate the cube points using the function:

```

def cube_points(c,wid):
    """ Creates a list of points for plotting
        a cube with plot. (the first 5 points are
        the bottom square, some sides repeated). """
    p = []
    # bottom
    p.append([c[0]-wid,c[1]-wid,c[2]-wid])
    p.append([c[0]-wid,c[1]+wid,c[2]-wid])
    p.append([c[0]+wid,c[1]+wid,c[2]-wid])
    p.append([c[0]+wid,c[1]-wid,c[2]-wid])
    p.append([c[0]-wid,c[1]-wid,c[2]-wid]) # same as first to close plot

    # top
    p.append([c[0]-wid,c[1]-wid,c[2]+wid])
    p.append([c[0]-wid,c[1]+wid,c[2]+wid])
    p.append([c[0]+wid,c[1]+wid,c[2]+wid])
    p.append([c[0]+wid,c[1]-wid,c[2]+wid])
    p.append([c[0]-wid,c[1]-wid,c[2]+wid]) # same as first to close plot

    # vertical sides
    p.append([c[0]-wid,c[1]-wid,c[2]+wid])
    p.append([c[0]-wid,c[1]+wid,c[2]+wid])
    p.append([c[0]-wid,c[1]+wid,c[2]-wid])
    p.append([c[0]+wid,c[1]+wid,c[2]-wid])
    p.append([c[0]+wid,c[1]+wid,c[2]+wid])
    p.append([c[0]+wid,c[1]-wid,c[2]+wid])
    p.append([c[0]+wid,c[1]-wid,c[2]-wid])

    return array(p).T

```

Some points are reoccurring so that `plot()` will generate a nice-looking cube.

With a homography and a camera calibration matrix, we can now determine the relative transformation between the two views:

```

# camera calibration
K = my_calibration((747,1000))

# 3D points at plane z=0 with sides of length 0.2
box = cube_points([0,0,0.1],0.1)

# project bottom square in first image
cam1 = camera.Camera( hstack((K,dot(K,array([[0],[0],[-1]]))) )) )
# first points are the bottom square
box_cam1 = cam1.project(homography.make_homog(box[:, :5]))

# use H to transfer points to the second image
box_trans = homography.normalize(dot(H,box_cam1))

# compute second camera matrix from cam1 and H
cam2 = camera.Camera(dot(H,cam1.P))
A = dot(linalg.inv(K),cam2.P[:, :3])
A = array([A[:,0],A[:,1],cross(A[:,0],A[:,1])]).T
cam2.P[:, :3] = dot(K,A)

# project with the second camera
box_cam2 = cam2.project(homography.make_homog(box))

# test: projecting point on z=0 should give the same
point = array([1,1,0,1]).T
print homography.normalize(dot(dot(H,cam1.P),point))
print cam2.project(point)

```

Here we use a version of the image with resolution 747×1000 and first generate the calibration matrix for that image size. Next, points for a cube at the origin are created. The first five points generated by `cube_points()` correspond to the bottom, which in this case will lie on the plane defined by $Z = 0$, the plane of the marker. The first image (top left in Figure 4-4) is roughly a straight frontal view of the book and will be used as our template image. Since the scale of the scene coordinates is arbitrary, we create a first camera with matrix

$$P_1 = K \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{bmatrix},$$

which has coordinate axis aligned with the camera and placed above the marker. The first five 3D points are projected onto the image. With the estimated homography, we can transform these to the second image. Plotting them should show the corners at the same marker locations (see top right in Figure 4-4).

Now, composing P_1 with H as a camera matrix for the second image,

$$P_2 = H P_1,$$

will transform points on the marker plane $Z = 0$ correctly. This means that the first two columns and the fourth column of P_2 are correct. Since we know that the first 3×3 block should be $K R$ and R is a rotation matrix, we can recover the third column by

multiplying P_2 with the inverse of the calibration matrix and replacing the third column with the cross product of the first two.

As a sanity check, we can project a point on the marker plane with the new matrix and check that it gives the same projection as the same point transformed with the first camera and the homography. You should get the same printout in your console.

Visualizing the projected points can be done like this:

```
im0 = array(Image.open('book_frontal.JPG'))
im1 = array(Image.open('book_perspective.JPG'))

# 2D projection of bottom square
figure()
imshow(im0)
plot(box_cam1[0,:],box_cam1[1:],linewidth=3)

# 2D projection transferred with H
figure()
imshow(im1)
plot(box_trans[0,:],box_trans[1:],linewidth=3)

# 3D cube
figure()
imshow(im1)
plot(box_cam2[0,:],box_cam2[1:],linewidth=3)

show()
```

This should give three figures like the images in Figure 4-4. To be able to reuse these computations for future examples, we can save the camera matrices using Pickle:

```
import pickle

with open('ar_camera.pkl','w') as f:
    pickle.dump(K,f)
    pickle.dump(dot(linalg.inv(K),cam2.P),f)
```

Now we have seen how to compute the camera matrix given a planar scene object. We combined feature matching with homographies and camera calibration to produce a simple example of placing a cube in an image. With camera pose estimation, we now have the building blocks in place for creating simple augmented reality applications.

4.4 Augmented Reality

Augmented reality (AR) is a collective term for placing objects and information on top of image data. The classic example is placing a 3D computer graphics model so that it looks like it belongs in the scene, and moves naturally with the camera motion in the case of video. Given an image with a marker plane as in the section above, we can compute the camera's position and pose and use that to place computer graphics models so that they are rendered correctly. In this last section of our camera chapter we will show how to build a simple AR example. We will use two tools for this, PyGame and PyOpenGL.

PyGame and PyOpenGL

PyGame is a popular package for game development that easily handles display windows, input devices, events, and much more. PyGame is open source and available from <http://www.pygame.org/>. It is actually a Python binding for the SDL game engine. For installation instructions, see Appendix A. For more details on programming with PyGame, see, for example, [21].

PyOpenGL is the Python binding to the OpenGL graphics programming interface. OpenGL comes pre-installed on almost all systems and is a crucial part for graphics performance. OpenGL is cross platform and works the same across operating systems. Take a look at <http://www.opengl.org/> for more information on OpenGL. The getting started page (http://www.opengl.org/wiki/Getting_started) has resources for beginners. PyOpenGL is open source and easy to install; see Appendix A for details. More information can be found on the project website, <http://pyopengl.sourceforge.net/>.

There is no way we can cover any significant portion of OpenGL programming. We will instead just show the important parts, for example how to use camera matrices in OpenGL and setting up a basic 3D model. Some good examples and demos are available in the PyOpenGL-Demo package (<http://pypi.python.org/pypi/PyOpenGL-Demo>). This is a good place to start if you are new to PyOpenGL.

We want to place a 3D model in a scene using OpenGL. To use PyGame and PyOpenGL for this application, we need to import the following at the top of our scripts:

```
from OpenGL.GL import *
from OpenGL.GLU import *
import pygame, pygame.image
from pygame.locals import *
```

As you can see, we need two main parts from OpenGL. The GL part contains all functions starting with “gl”, which, you will see, are most of the ones we need. The GLU part is the OpenGL Utility library and contains some higher-level functionality. We will mainly use it to set up the camera projection. The `pygame` part sets up the window and event controls, and `pygame.image` is used for loading image and creating OpenGL textures. The `pygame.locals` is needed for setting up the display area for OpenGL.

The two main components of setting up an OpenGL scene are the projection and model view matrices. Let's get started and see how to create these matrices from our pin-hole cameras.

From Camera Matrix to OpenGL Format

OpenGL uses 4×4 matrices to represent transforms (both 3D transforms and projections). This is only slightly different from our use of 3×4 camera matrices. However, the camera-scene transformations are separated in two matrices, the `GL_PROJECTION` matrix and the `GL_MODELVIEW` matrix. `GL_PROJECTION` handles the image formation properties and is the equivalent of our internal calibration matrix K . `GL_MODELVIEW` handles the 3D transformation of the relation between the objects and the camera. This corresponds roughly to the R and t part of our camera matrix. One difference is that the coordinate system is assumed to be centered at the camera so the

GL_MODELVIEW matrix actually contains the transformation that places the objects in front of the camera. There are many peculiarities with working in OpenGL; we will comment on them as they are encountered in the examples below.

Given that we have a camera calibrated so that the calibration matrix K is known, the following function translates the camera properties to an OpenGL projection matrix:

```
def set_projection_from_camera(K):
    """ Set view from a camera calibration matrix. """

    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()

    fx = K[0,0]
    fy = K[1,1]
    fovy = 2*arctan(0.5*height/fy)*180/pi
    aspect = (width*fy)/(height*fx)

    # define the near and far clipping planes
    near = 0.1
    far = 100.0

    # set perspective
    gluPerspective(fovy,aspect,near,far)
    glViewport(0,0,width,height)
```

We assume the calibration to be of the simpler form in (4.3) with the optical center at the image center. The first function `glMatrixMode()` sets the working matrix to `GL_PROJECTION` and subsequent commands will modify this matrix.¹ Then `glLoadIdentity()` sets the matrix to the identity matrix, basically resetting any prior changes. We then calculate the vertical field of view in degrees with the help of the image height and the camera's focal length as well as the aspect ratio. An OpenGL projection also has a near and far clipping plane to limit the depth range of what is rendered. We just set the near depth to be small enough to contain the nearest object and the far depth to some large number. We use the GLU utility function `gluPerspective()` to set the projection matrix and define the whole image to be the view port (essentially what is to be shown). There is also an option to load a full projection matrix with `glLoadMatrixf()` similar to the model view function below. This is useful when the simple version of the calibration matrix is not good enough.

The model view matrix should encode the relative rotation and translation that brings the object in front of the camera (as if the camera was at the origin). It is a 4×4 matrix that typically looks like this:

$$\begin{bmatrix} R & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix},$$

where R is a rotation matrix with columns equal to the direction of the three coordinate axis and $\mathbf{t}$ is a translation vector. When creating a model view matrix, the rotation part

¹ This is an odd way to handle things, but there are only two matrices to switch between, `GL_PROJECTION` and `GL_MODELVIEW`, so it is manageable.

will need to hold all rotations (object and coordinate system) by multiplying together the individual components.

The following function shows how to take a 3×4 pin-hole camera matrix with the calibration removed (multiply P with K^{-1}) and create a model view:

```
def set_modelview_from_camera(Rt):
    """ Set the model view matrix from camera pose. """

    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()

    # rotate teapot 90 deg around x-axis so that z-axis is up
    Rx = array([[1,0,0],[0,0,-1],[0,1,0]])

    # set rotation to best approximation
    R = Rt[:, :3]
    U,S,V = linalg.svd(R)
    R = dot(U,V)
    R[0,:] = -R[0,:] # change sign of x-axis

    # set translation
    t = Rt[:, 3]

    # setup 4*4 model view matrix
    M = eye(4)
    M[:3,:3] = dot(R,Rx)
    M[:3,3] = t

    # transpose and flatten to get column order
    M = M.T
    m = M.flatten()

    # replace model view with the new matrix
    glLoadMatrixf(m)
```

First, we switch to work on the GL_MODELVIEW matrix and reset it. Then we create a 90-degree rotation matrix, since the object we want to place needs to be rotated (you will see below). Then we make sure that the rotation part of the camera matrix is indeed a rotation matrix, in case there are errors or noise when we estimated the camera matrix. This is done with SVD and the best rotation matrix approximation is given by $R = UV^T$. The OpenGL coordinate system is a little different, so we flip the x-axis around. Then we set the model view matrix M by multiplying the rotations. The function `glLoadMatrixf()` sets the model view matrix and takes an array of the 16 values of the matrix taken *column-wise*. Transposing and then flattening accomplishes this.

Placing Virtual Objects in the Image

The first thing we need to do is to add the image (the one we want to place virtual objects in) as a background. In OpenGL this is done by creating a quadrilateral, a *quad*, that fills the whole view. The easiest way to do this is to draw the quad with the projection and model view matrices reset so that the coordinates go from -1 to 1 in each dimension.

This function loads an image, converts it to an OpenGL texture, and places that texture on the quad:

```
def draw_background(imname):
    """ Draw background image using a quad. """

    # load background image (should be .bmp) to OpenGL texture
    bg_image = pygame.image.load(imname).convert()
    bg_data = pygame.image.tostring(bg_image, "RGBX", 1)

    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    # bind the texture
    glEnable(GL_TEXTURE_2D)
    glBindTexture(GL_TEXTURE_2D, glGenTextures(1))
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA, GL_UNSIGNED_BYTE, bg_data)
    glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST)
    glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST)

    # create quad to fill the whole window
    glBegin(GL_QUADS)
    glTexCoord2f(0.0, 0.0); glVertex3f(-1.0, -1.0, -1.0)
    glTexCoord2f(1.0, 0.0); glVertex3f( 1.0, -1.0, -1.0)
    glTexCoord2f(1.0, 1.0); glVertex3f( 1.0,  1.0, -1.0)
    glTexCoord2f(0.0, 1.0); glVertex3f(-1.0,  1.0, -1.0)
    glEnd()

    # clear the texture
    glDeleteTextures(1)
```

This function first uses some PyGame functions to load an image and serialize it to a raw string representation that can be used by PyOpenGL. Then we reset the model view and clear the color and depth buffer. Next, we bind the texture so that we can use it for the quad and specify interpolation. The quad is defined with corners at -1 and 1 in both dimensions. Note that the coordinates in the texture image go from 0 to 1 . Finally, we clear the texture so it doesn't interfere with what we want to draw later.

Now we are ready to place objects in the scene. We will use the “hello world” computer graphics example, the Utah teapot (http://en.wikipedia.org/wiki/Utah_teapot). This teapot has a rich history and is available as one of the standard shapes in GLUT:

```
from OpenGL.GLUT import *
glutSolidTeapot(size)
```

This generates a solid teapot model of relative size *size*.

The following function will set up the color and properties to make a pretty red teapot:

```
def draw_teapot(size):
    """ Draw a red teapot at the origin. """
    glEnable(GL_LIGHTING)
    glEnable(GL_LIGHT0)
    glEnable(GL_DEPTH_TEST)
    glClear(GL_DEPTH_BUFFER_BIT)
```

```

# draw red teapot
glMaterialfv(GL_FRONT, GL_AMBIENT, [0,0,0,0])
glMaterialfv(GL_FRONT, GL_DIFFUSE, [0.5,0.0,0.0,0.0])
glMaterialfv(GL_FRONT, GL_SPECULAR, [0.7,0.6,0.6,0.0])
glMaterialf(GL_FRONT, GL_SHININESS, 0.25*128.0)
glutSolidTeapot(size)

```

The first two lines enable lighting and a light. Lights are numbered as `GL_LIGHT0`, `GL_LIGHT1`, etc. We will only use one light in this example. The `glEnable()` function is used to turn on OpenGL features. These are defined with uppercase constants. Turning off a feature is done with the corresponding function `glDisable()`. Next, depth testing is turned on so that objects are rendered according to their depth (so that far-away objects are not drawn in front of near objects) and the depth buffer is cleared. Next, the material properties of the object, such as the diffuse and specular colors, are specified. The last line adds a solid Utah teapot with the specified material properties.

Tying It All Together

The full script for generating an image like the one in Figure 4-5 looks like this (assuming that you also have the functions introduced above in the same file):

```

from OpenGL.GL import *
from OpenGL.GLU import *
from OpenGL.GLUT import *
import pygame, pygame.image
from pygame.locals import *
import pickle

width,height = 1000,747

def setup():
    """ Setup window and pygame environment. """
    pygame.init()
    pygame.display.set_mode((width,height),OPENGL | DOUBLEBUF)
    pygame.display.set_caption('OpenGL AR demo')

# load camera data
with open('ar_camera.pkl','r') as f:
    K = pickle.load(f)
    Rt = pickle.load(f)

setup()
draw_background('book_perspective.bmp')
set_projection_from_camera(K)
set_modelview_from_camera(Rt)
draw_teapot(0.02)

while True:
    event = pygame.event.poll()
    if event.type in (QUIT,KEYDOWN):
        break
    pygame.display.flip()

```




Figure 4-5. Augmented reality. Placing a computer graphics model on a book in a scene using camera parameters computed from feature matches: the Utah teapot rendered in place aligned with the coordinate axis (top); sanity check to see the position of the origin (bottom).

First, this script loads the camera calibration matrix and the rotation and translation part of the camera matrix using Pickle. This assumes that you saved them as described on page 89. The `setup()` function initializes PyGame, sets the window to the size of the image, and makes the drawing area a double buffer OpenGL window. Next, the background image is loaded and placed to fit the window. The camera and model view matrices are set and finally the teapot is drawn at the correct position.

Events in PyGame are handled using infinite loops with regular polling for any changes. These can be keyboard, mouse, or other events. In this case, we check if the application was quit or if a key was pressed and exit the loop. The command `pygame.display.flip()` draws the objects on the screen.

The result should look like Figure 4-5. As you can see, the orientation is correct (the teapot is aligned with the sides of the cube in Figure 4-4). To check that the placement is correct, you can try to make the teapot really small by passing a smaller value for the *size* variable. The teapot should be placed close to the [0, 0, 0] corner of the cube in Figure 4-4. An example is shown in Figure 4-5.

Loading Models

Before we end this chapter, we will touch upon one last detail: loading 3D models and displaying them. The PyGame cookbook has a script for loading models in .obj format available at <http://www.pygame.org/wiki/OBJFileLoader>. You can learn more about the .obj format and the corresponding material file format at http://en.wikipedia.org/wiki/Wavefront_.obj_file.

Let's see how to use that with a basic example. We will use a freely available toy plane model from <http://www.oyonale.com/modeles.php>.² Download the .obj version and save it as *toyplane.obj*. You can, of course, replace this model with any model of your choice; the code below will be the same.

Assuming that you downloaded the file as *objloader.py*, add the following function to the file you used for the teapot example above:

```
def load_and_draw_model(filename):
    """ Loads a model from an .obj file using objloader.py.
        Assumes there is a .mtl material file with the same name. """
    glEnable(GL_LIGHTING)
    glEnable(GL_LIGHT0)
    glEnable(GL_DEPTH_TEST)
    glClear(GL_DEPTH_BUFFER_BIT)

    # set model color
    glMaterialfv(GL_FRONT, GL_AMBIENT, [0,0,0,0])
    glMaterialfv(GL_FRONT, GL_DIFFUSE, [0.5,0.75,1.0,0.0])
    glMaterialf(GL_FRONT, GL_SHININESS, 0.25*128.0)

    # load from a file
    import objloader
    obj = objloader.OBJ(filename, swapyz=True)
    glCallList(obj.gl_list)
```

Same as before, we set the lighting and the color properties of the model. Next, we load a model file into an OBJ object and execute the OpenGL calls from the file.

You can set the texture and material properties in a corresponding .mtl file. The *objloader* module actually requires a material file. Rather than modifying the loading script, we take the pragmatic approach of just creating a tiny material file. In this case, we'll just specify the color.

² Models courtesy of Gilles Tran (Creative Commons License By Attribution).

Create a file *toyplane.mtl* with the following lines:

```
newmtl lightblue
Kd 0.5 0.75 1.0
illum 1
```

This sets the diffuse color of the object to a light grayish blue. Now, make sure to replace the “usmtl” tag in your .obj file with

```
usmtl lightblue
```

Adding textures we leave to the exercises. Replacing the call to `draw_teapot()` in the example above with

```
load_and_draw_model('toyplane.obj')
```

should generate a window like the one shown in Figure 4-6.

This is as deep as we will go into augmented reality and OpenGL in this book. With the recipe for calibrating cameras, computing camera pose, translating the cameras into OpenGL format, and rendering models in the scene, the groundwork is laid for you to continue exploring augmented reality. In the next chapter, we will continue with the camera model and compute 3D structure and camera pose without the use of markers.

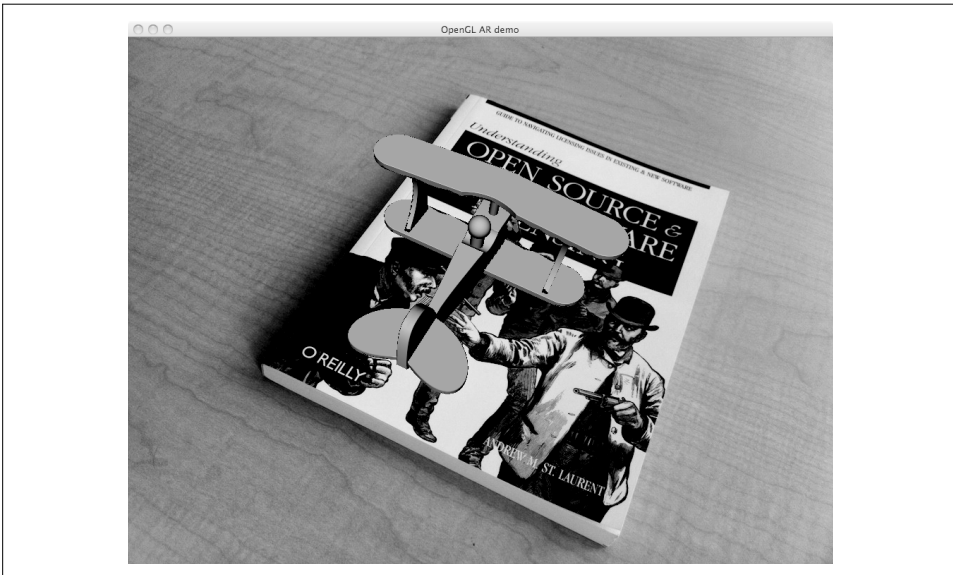


Figure 4-6. Loading a 3D model from an .obj file and placing it on a book in a scene using camera parameters computed from feature matches.

Exercises

1. Modify the example code for the motion in Figure 4-2 to transform the points instead of the camera. You should get the same plot. Experiment with different transformations and plot the results.
2. Some of the Oxford multi-view datasets have camera matrices given. Compute the camera positions for one of the sets and plot the camera path. Does it match with what you are seeing in the images?
3. Take some images of a scene with a planar marker or object. Match features to a full-frontal image to compute the pose of each image's camera location. Plot the camera trajectory and the plane of the marker. Add the feature points if you like.
4. In our augmented reality example, we assumed the object to be placed at the origin and applied only the camera's position to the model view matrix. Modify the example to place several objects at different locations by adding the object transformation to the matrix. For example, place a grid of teapots on the marker.
5. Take a look at the online documentation for .obj model files and see how to use textured models. Find a model (or create your own) and add it to the scene.

Multiple View Geometry

This chapter will show you how to handle multiple views and how to use the geometric relationships between them to recover camera positions and 3D structure. With images taken at different view points, it is possible to compute 3D scene points as well as camera locations from feature matches. We introduce the necessary tools and show a complete 3D reconstruction example. The last part of the chapter shows how to compute dense depth reconstructions from stereo images.

5.1 Epipolar Geometry

Multiple view geometry is the field studying the relationship between cameras and features when there are correspondences between many images that are taken from varying viewpoints. The image features are usually interest points, and we will focus on that case throughout this chapter. The most important constellation is two-view geometry.

With two views of a scene and corresponding points in these views, there are geometric constraints on the image points as a result of the relative orientation of the cameras, the properties of the cameras, and the position of the 3D points. These geometric relationships are described by what is called *epipolar geometry*. This section will give a very short description of the basic components we need. For more details on the subject, see [13].

Without any prior knowledge of the cameras, there is an inherent ambiguity in that a 3D point, $\mathbf{X}$, transformed with an arbitrary (4×4) homography H as $H\mathbf{X}$ will have the same image point in a camera $P H^{-1}$ as the original point in the camera P . Expressed with the camera equation, this is

$$\lambda \mathbf{x} = P\mathbf{X} = P H^{-1} H\mathbf{X} = \hat{P}\hat{\mathbf{X}}.$$

Because of this ambiguity, when analyzing two view geometry we can always transform the cameras with a homography to simplify matters. Often this homography is just a

rigid transformation to change the coordinate system. A good choice is to set the origin and coordinate axis to align with the first camera so that

$$P_1 = K_1[I \mid 0] \quad \text{and} \quad P_2 = K_2[R \mid \mathbf{t}].$$

Here we use the same notation as in Chapter 4; K_1 and K_2 are the calibration matrices, R is the rotation of the second camera, and $\mathbf{t}$ is the translation of the second camera. Using these camera matrices, one can derive a condition for the projection of a point $\mathbf{X}$ to image points $\mathbf{x}_1$ and $\mathbf{x}_2$ (with P_1 and P_2 , respectively). This condition is what makes it possible to recover the camera matrices from corresponding image points.

The following equation must be satisfied:

$$\mathbf{x}_2^T F \mathbf{x}_1 = 0, \quad (5.1)$$

where

$$F = K_2^{-T} S_{\mathbf{t}} R K_1^{-1},$$

and the matrix $S_{\mathbf{t}}$ is the skew symmetric matrix

$$S_{\mathbf{t}} = \begin{bmatrix} 0 & -t_3 & t_2 \\ t_3 & 0 & -t_1 \\ -t_2 & t_1 & 0 \end{bmatrix}. \quad (5.2)$$

Equation (5.1) is called the *epipolar constraint*. The matrix F in the epipolar constraint is called the *fundamental matrix* and as you can see, it is expressed in components of the two camera matrices (their relative rotation R and translation $\mathbf{t}$). The fundamental matrix has rank 2 and $\det(F) = 0$. This will be used in algorithms for estimating F .

The equations above mean that the camera matrices can be recovered from F , which in turn can be computed from point correspondences, as we will see later. Without knowing the internal calibration (K_1 and K_2), the camera matrices are only recoverable up to a projective transformation. With known calibration, the reconstruction will be metric. A *metric reconstruction* is a 3D reconstruction that correctly represents distances and angles.¹

There is one final piece of geometry needed before we can proceed to actually using this theory on some image data. Given a point in one of the images, for example $\mathbf{x}_2$ in the second view, equation (5.1) defines a line in the first image since

$$\mathbf{x}_2^T F \mathbf{x}_1 = \mathbf{l}_1^T \mathbf{x}_1 = 0.$$

The equation $\mathbf{l}_1^T \mathbf{x}_1 = 0$ determines a line with all points $\mathbf{x}_1$ in the first image satisfying the equation belonging to the line. This line is called an *epipolar line* corresponding to the point $\mathbf{x}_2$. This means that a corresponding point to $\mathbf{x}_2$ must lie on this line. The fundamental matrix can therefore help the search for correspondences by restricting the search to this line.

¹ The absolute scale of the reconstruction cannot be recovered, but that is rarely a problem.

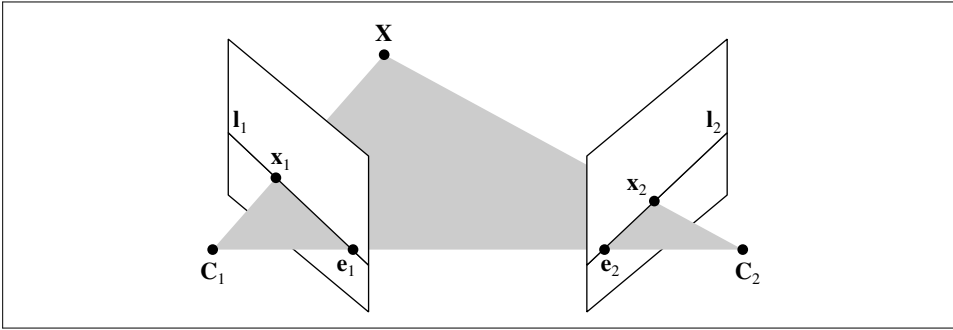


Figure 5-1. An illustration of epipolar geometry. A 3D point X is projected to x_1 and x_2 , in the two views, respectively. The baseline between the two camera centers, C_1 and C_2 , intersect the image planes in the epipoles, e_1 and e_2 . The lines l_1 and l_2 are called epipolar lines.

The epipolar lines all meet in a point, e , called the *epipole*. The epipole is actually the image point corresponding to the projection of the other camera center. This point can be outside the actual image, depending on the relative orientation of the cameras. Since the epipole lies on all epipolar lines, it must satisfy $F e_1 = 0$. It can, therefore, be computed as the null vector of F , as we will see later. The other epipole can be computed from the relation $e_2^T F = 0$. The epipoles and the epipolar lines are illustrated in Figure 5-1.

A Sample Data Set

In the coming sections, we will need a data set with image points, 3D points, and camera matrices to experiment with and illustrate the algorithms. We will use one of the sets from the Oxford multi-view datasets available at <http://www.robots.ox.ac.uk/~vgg/data/data-mview.html>. Download the zipped file for the Merton1 data. The following script will load all the data for you:

```
import camera

# load some images
im1 = array(Image.open('images/001.jpg'))
im2 = array(Image.open('images/002.jpg'))

# load 2D points for each view to a list
points2D = [loadtxt('2D/00'+str(i+1)+'.corners').T for i in range(3)]

# load 3D points
points3D = loadtxt('3D/p3d').T

# load correspondences
corr = genfromtxt('2D/nview-corners', dtype='int', missing='*')

# load cameras to a list of Camera objects
P = [camera.Camera(loadtxt('2D/00'+str(i+1)+'.P')) for i in range(3)]
```

This will load the first two images (out of three), all the image feature points² for the three views, the reconstructed 3D points corresponding to the image points, which points correspond across views, and finally the camera matrices (where we used the Camera class from the previous chapter). Here we used `loadtxt()` to read the text files to NumPy arrays. The correspondences contain missing data, since not all points are visible or successfully matched in all views. The correspondences need to be loaded with this taken into account. The function `genfromtxt()` solves this by replacing the missing values (denoted with “*” in this file) with -1 .

A convenient way of running this script and getting all the data is to save the code above in a file, for example `load_vggdata.py`, and use the command `execfile()` at the beginning of your scripts or experiments:

```
execfile('load_vggdata.py')
```

Let’s see what this data looks like. Try to project the 3D points into one view and compare the results with the observed image points:

```
# make 3D points homogeneous and project
X = vstack( (points3D,ones(points3D.shape[1])) )
x = P[0].project(X)

# plotting the points in view 1
figure()
imshow(im1)
plot(points2D[0][0],points2D[0][1], '*')
axis('off')

figure()
imshow(im1)
plot(x[0],x[1], 'r.')
axis('off')

show()
```

This creates a plot with the first view and image points in that view; for comparison, the projected points are shown in a separate figure. Figure 5-2 shows the resulting plots. If you look closely, you will see that the second plot with the projected 3D points contains more points than the first. These are image feature points reconstructed from view 2 and 3 but not detected in view 1.

Plotting 3D Data with Matplotlib

To visualize our 3D reconstructions, we need to be able to plot in 3D. The `mplot3d` toolkit in `Matplotlib` provides 3D plotting of points, lines, contours, surfaces and most other basic plotting components as well as 3D rotation and scaling from the controls of the figure window.

² Actually Harris corner points; see Section 2.1.

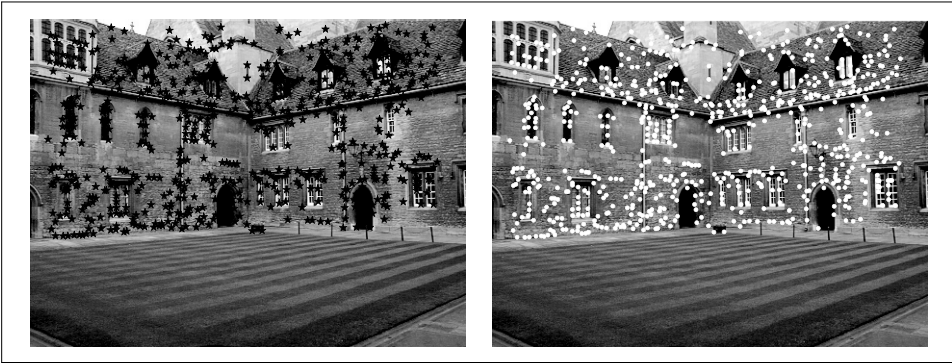


Figure 5-2. The Merton1 data set from the Oxford multi-view datasets: view 1 with image points shown (left); view 1 with projected 3D points (right).

Making a plot in 3D is done by adding the `projection="3d"` keyword to the axes object like this:

```
from mpl_toolkits.mplot3d import axes3d

fig = figure()
ax = fig.gca(projection="3d")

# generate 3D sample data
X,Y,Z = axes3d.get_test_data(0.25)

# plot the points in 3D
ax.plot(X.flatten(),Y.flatten(),Z.flatten(),'o')

show()
```

The function `get_test_data()` generates sample points on a regular x, y grid with the parameter determining the spacing. Flattening these grids gives three lists of points that can be sent to `plot()`. This should plot 3D points on what looks like a surface. Try it out and see for yourself.

Now we can plot the Merton sample data to see what the 3D points look like:

```
# plotting 3D points
from mpl_toolkits.mplot3d import axes3d
fig = figure()
ax = fig.gca(projection='3d')
ax.plot(points3D[0],points3D[1],points3D[2], 'k.')
```

Figure 5-3 shows the 3D points from three different views. The figure window and controls look like the standard plot windows for images and 2D data with an additional 3D rotation tool.

Computing F—The Eight Point Algorithm

The *eight point algorithm* is an algorithm for computing the fundamental matrix from point correspondences. Here's a brief description; the details can be found in [14] and [13].

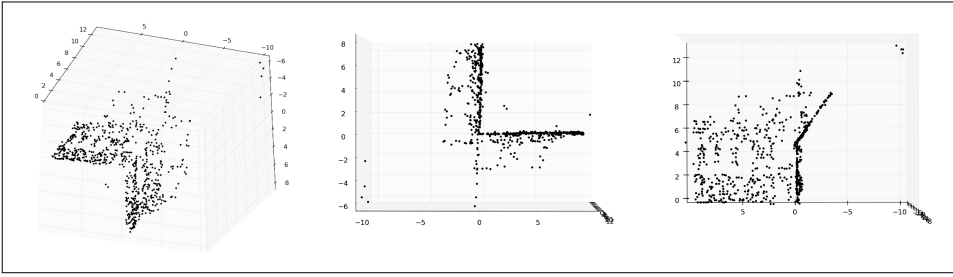


Figure 5-3. The 3D points of the Merton1 data set from the Oxford multi-view datasets shown using Matplotlib: view from above and to the side (left); view from the top, showing the building walls and points on the roof (middle); side view showing the profile of one of the walls and a frontal view of points on the other wall (right).

The epipolar constraint (5.1) can be written as a linear system like

$$\begin{bmatrix} x_2^1 x_1^1 & x_2^1 y_1^1 & x_2^1 w_1^1 & \dots & w_2^1 w_1^1 \\ x_2^2 x_1^2 & x_2^2 y_1^2 & x_2^2 w_1^2 & \dots & w_2^2 w_1^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_2^n x_1^n & x_2^n y_1^n & x_2^n w_1^n & \dots & w_2^n w_1^n \end{bmatrix} \begin{bmatrix} F_{11} \\ F_{12} \\ F_{13} \\ \vdots \\ F_{33} \end{bmatrix} = \mathbf{A}\mathbf{f} = 0,$$

where $\mathbf{f}$ contains the elements of F , $\mathbf{x}_1^i = [x_1^i, y_1^i, w_1^i]$ and $\mathbf{x}_2^i = [x_2^i, y_2^i, w_2^i]$ is a correspondence pair, and there are n point correspondences in total. The fundamental matrix has nine elements, but since the scale is arbitrary, only eight equations are needed. Eight point correspondences are therefore needed to compute F ; hence the name of the algorithm.

Create a file `sfm.py`, and add the following function for the eight point algorithm that minimizes $\|\mathbf{A}\mathbf{f}\|$:

```
def compute_fundamental(x1,x2):
    """ Computes the fundamental matrix from corresponding points
        (x1,x2 3*n arrays) using the normalized 8 point algorithm.
        each row is constructed as
        [x'*x, x'*y, x', y'*x, y'*y, y', x, y, 1] """

    n = x1.shape[1]
    if x2.shape[1] != n:
        raise ValueError("Number of points don't match.")

    # build matrix for equations
    A = zeros((n,9))
    for i in range(n):
        A[i] = [x1[0,i]*x2[0,i], x1[0,i]*x2[1,i], x1[0,i]*x2[2,i],
                x1[1,i]*x2[0,i], x1[1,i]*x2[1,i], x1[1,i]*x2[2,i],
                x1[2,i]*x2[0,i], x1[2,i]*x2[1,i], x1[2,i]*x2[2,i] ]

    # compute linear least square solution
    U,S,V = linalg.svd(A)
    F = V[-1].reshape(3,3)
```

```

# constrain F
# make rank 2 by zeroing out last singular value
U,S,V = linalg.svd(F)
S[2] = 0
F = dot(U,dot(diag(S),V))

return F

```

As usual, we compute the least squares solution using SVD. Since the resulting solution might not have rank 2 as a proper fundamental matrix should, we replace the result with the closest rank 2 approximation by zeroing out the last singular value. This is a standard trick and a useful one to know. The function ignores the important step of normalizing the image coordinates. Ignoring normalization could give numerical problems. Let's leave that for later.

The Epipole and Epipolar Lines

As mentioned at the start of this section, the epipole satisfies $F\mathbf{e}_1 = 0$ and can be computed from the null space of F . Add this function to *sfm.py*:

```

def compute_epipole(F):
    """ Computes the (right) epipole from a
        fundamental matrix F.
        (Use with F.T for left epipole.) """

    # return null space of F (Fx=0)
    U,S,V = linalg.svd(F)
    e = V[-1]
    return e/e[2]

```

If you want the epipole corresponding to the left null vector (corresponding to the epipole in the other image), just transpose F before passing it as input.

We can try these two functions on the first two views of our sample data set like this:

```

import sfm

# index for points in first two views
ndx = (corr[:,0]>=0) & (corr[:,1]>=0)

# get coordinates and make homogeneous
x1 = points2D[0][:,corr[ndx,0]]
x1 = vstack( (x1,ones(x1.shape[1])) )
x2 = points2D[1][:,corr[ndx,1]]
x2 = vstack( (x2,ones(x2.shape[1])) )

# compute F
F = sfm.compute_fundamental(x1,x2)

# compute the epipole
e = sfm.compute_epipole(F)

# plotting
figure()
imshow(im1)

```

```

# plot each line individually, this gives nice colors
for i in range(5):
    sfm.plot_epipolar_line(im1,F,x2[:,i],e,False)
axis('off')

figure()
imshow(im2)
# plot each point individually, this gives same colors as the lines
for i in range(5):
    plot(x2[0,i],x2[1,i],'o')
axis('off')

show()

```

First, the points that are in correspondence between the two images are selected and made into homogeneous coordinates. Here we just read them from a text file; in reality these would be the result of extracting features and matching them as we did in Chapter 2. The missing values in the correspondence list *corr* are -1 , so picking indices greater or equal to zero gives the points visible in each view. The two conditions are combined with the array operator `&`.

Finally, the first five of the epipolar lines are shown in the first view and the corresponding matching points in view 2. Here we used the helper plot function:

```

def plot_epipolar_line(im,F,x,epipole=None,show_epipole=True):
    """ Plot the epipole and epipolar line  $F \cdot x = 0$ 
        in an image.  $F$  is the fundamental matrix
        and  $x$  a point in the other image. """

    m,n = im.shape[:2]
    line = dot(F,x)

    # epipolar line parameter and values
    t = linspace(0,n,100)
    lt = array([(line[2]+line[0]*tt)/(-line[1]) for tt in t])

    # take only line points inside the image
    ndx = (lt>=0) & (lt<m)
    plot(t[ndx],lt[ndx],linewidth=2)

    if show_epipole:
        if epipole is None:
            epipole = compute_epipole(F)
        plot(epipole[0]/epipole[2],epipole[1]/epipole[2], 'r*')

```

This function parameterizes the line with the range of the x axis and removes parts of lines above and below the image border. If the last parameter *show_epipole* is true, the epipole will be plotted as well (and computed if not passed as input). The plots are shown in Figure 5-4. The color coding matches between the plots so you can see that the corresponding point in one image lies somewhere along the same-color line as a point in the other image.



Figure 5-4. Epipolar lines in view 1 shown for five points in view 2 of the Merton1 data. The bottom row shows a closeup of the area around the points. The lines can be seen to converge on a point outside the image to the left. The lines show where point correspondences can be found in the other image (the color coding matches between lines and points).

5.2 Computing with Cameras and 3D Structure

The previous section covered relationships between views and how to compute the fundamental matrix and epipolar lines. Here we briefly explain the tools we need for computing with cameras and 3D structure.

Triangulation

Given known camera matrices, a set of point correspondences can be triangulated to recover the 3D positions of these points. The basic algorithm is fairly simple.

For two views with camera matrices P_1 and P_2 , each with a projection $\mathbf{x}_1$ and $\mathbf{x}_2$ of the same 3D point $\mathbf{X}$ (all in homogeneous coordinates), the camera equation (4.1) gives the relation

$$\begin{bmatrix} P_1 & -\mathbf{x}_1 & 0 \\ P_2 & 0 & -\mathbf{x}_2 \end{bmatrix} \begin{bmatrix} \mathbf{X} \\ \lambda_1 \\ \lambda_2 \end{bmatrix} = 0.$$

There might not be an exact solution to these equations due to image noise, errors in the camera matrices, or other sources of errors. Using SVD, we can get a least squares estimate of the 3D point.

Add the following function that computes the *least squares triangulation* of a point pair to *sfm.py*:

```
def triangulate_point(x1,x2,P1,P2):
    """ Point pair triangulation from
        least squares solution. """

    M = zeros((6,6))
    M[:3,:4] = P1
    M[3,:4] = P2
    M[:3,4] = -x1
    M[3,5] = -x2

    U,S,V = linalg.svd(M)
    X = V[-1,:4]

    return X / X[3]
```

The first four values in the last eigenvector are the 3D coordinates in homogeneous coordinates. To triangulate many points, we can add the following convenience function:

```
def triangulate(x1,x2,P1,P2):
    """ Two-view triangulation of points in
        x1,x2 (3*n homog. coordinates). """

    n = x1.shape[1]
    if x2.shape[1] != n:
        raise ValueError("Number of points don't match.")

    X = [ triangulate_point(x1[:,i],x2[:,i],P1,P2) for i in range(n)]
    return array(X).T
```

This function takes two arrays of points and returns an array of 3D coordinates.

Try the triangulation on the Merton1 data like this:

```
import sfm

# index for points in first two views
ndx = (corr[:,0]>=0) & (corr[:,1]>=0)

# get coordinates and make homogeneous
x1 = points2D[0][:,corr[ndx,0]]
x1 = vstack( (x1,ones(x1.shape[1])) )
x2 = points2D[1][:,corr[ndx,1]]
x2 = vstack( (x2,ones(x2.shape[1])) )

Xtrue = points3D[:,ndx]
Xtrue = vstack( (Xtrue,ones(Xtrue.shape[1])) )

# check first 3 points
Xest = sfm.triangulate(x1,x2,P[0].P,P[1].P)
print Xest[:, :3]
print Xtrue[:, :3]
```

```
# plotting
from mpl_toolkits.mplot3d import axes3d
fig = figure()
ax = fig.gca(projection='3d')
ax.plot(Xest[0],Xest[1],Xest[2],'ko')
ax.plot(Xtrue[0],Xtrue[1],Xtrue[2],'r.')
axis('equal')

show()
```

This will triangulate the points in correspondence from the first two views and print out the coordinates of the first three points to the console before plotting the recovered 3D points next to the true values. The printout looks like this:

```
[[ 1.03743725  1.56125273  1.40720017]
 [-0.57574987 -0.55504127 -0.46523952]
 [ 3.44173797  3.44249282  7.53176488]
 [ 1.          1.          1.         ]]
[[ 1.0378863  1.5606923  1.4071907 ]
 [-0.54627892 -0.5211711 -0.46371818]
 [ 3.4601538  3.4636809  7.5323397 ]
 [ 1.          1.          1.         ]]
```

The estimated points are close enough. The plot looks like Figure 5-5; as you can see, the points match fairly well.

Computing the Camera Matrix from 3D Points

With known 3D points and their image projections, the camera matrix, P , can be computed using a direct linear transform approach. This is essentially the inverse problem to triangulation and is sometimes called *camera resectioning*. This way to recover the camera matrix is again a least squares approach.

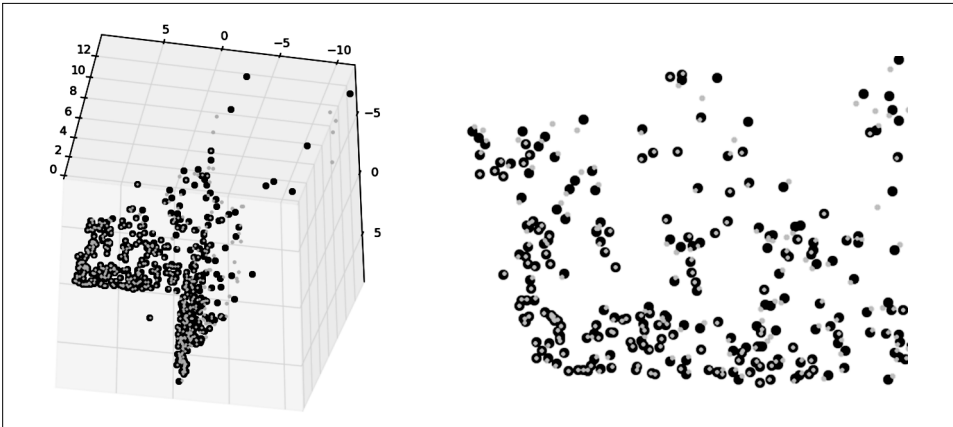


Figure 5-5. Triangulated points using camera matrices and point correspondences. The estimated points are shown with dark circles and the true points with light dots. View from above and to the side (left). Closeup of the points from one of the building walls (right).

From the camera equation (4.1), each visible 3D point $\mathbf{X}_i$ (in homogeneous coordinates) is projected to an image point $\mathbf{x}_i = [x_i, y_i, 1]$ as $\lambda_i \mathbf{x}_i = P \mathbf{X}_i$ and the corresponding points satisfy the relation

$$\begin{bmatrix} \mathbf{X}_1^T & 0 & 0 & -x_1 & 0 & 0 & \dots \\ 0 & \mathbf{X}_1^T & 0 & -y_1 & 0 & 0 & \dots \\ 0 & 0 & \mathbf{X}_1^T & -1 & 0 & 0 & \dots \\ \mathbf{X}_2^T & 0 & 0 & 0 & -x_2 & 0 & \dots \\ 0 & \mathbf{X}_2^T & 0 & 0 & -y_2 & 0 & \dots \\ 0 & 0 & \mathbf{X}_2^T & 0 & -1 & 0 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix} \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \\ \mathbf{p}_3^T \\ \lambda_1 \\ \lambda_2 \\ \vdots \end{bmatrix} = 0,$$

where $\mathbf{p}_1$, $\mathbf{p}_2$, and $\mathbf{p}_3$ are the three rows of P . This can be written more compactly as

$$M\mathbf{v} = 0.$$

The estimation of the camera matrix is then obtained using SVD. With the matrices described above, the code is straightforward. Add the function below to *sfm.py*:

```
def compute_P(x,X):
    """ Compute camera matrix from pairs of
        2D-3D correspondences (in homog. coordinates). """

    n = x.shape[1]
    if X.shape[1] != n:
        raise ValueError("Number of points don't match.")

    # create matrix for DLT solution
    M = zeros((3*n,12+n))
    for i in range(n):
        M[3*i,0:4] = X[:,i]
        M[3*i+1,4:8] = X[:,i]
        M[3*i+2,8:12] = X[:,i]
        M[3*i:3*i+3,i+12] = -x[:,i]

    U,S,V = linalg.svd(M)

    return V[-1,:12].reshape((3,4))
```

This function takes the image points and 3D points and builds up the matrix M above. The first 12 values of the last eigenvector are the elements of the camera matrix and are returned after a reshaping operation.

Again, let's try this on our sample data set. The following script will pick out the points that are visible in the first view (using the missing values from the correspondence list), make them into homogeneous coordinates, and estimate the camera matrix:

```
import sfm, camera

corr = corr[:,0] # view 1
ndx3D = where(corr>=0)[0] # missing values are -1
ndx2D = corr[ndx3D]
```



```

# select visible points and make homogeneous
x = points2D[0][:,ndx2D] # view 1
x = vstack( (x,ones(x.shape[1])) )
X = points3D[:,ndx3D]
X = vstack( (X,ones(X.shape[1])) )

# estimate P
Pest = camera.Camera(sfm.compute_P(x,X))

# compare!
print Pest.P / Pest.P[2,3]
print P[0].P / P[0].P[2,3]

xest = Pest.project(X)

# plotting
figure()
imshow(im1)
plot(x[0],x[1],'bo')
plot(xest[0],xest[1],'r.')
axis('off')

show()

```

To check the camera matrices, they are printed to the console in normalized form (by dividing with the last element). The printout looks like this:

```

[[ 1.06520794e+00 -5.23431275e+01  2.06902749e+01  5.08729305e+02]
 [-5.05773115e+01 -1.33243276e+01 -1.47388537e+01  4.79178838e+02]
 [ 3.05121915e-03 -3.19264684e-02 -3.43703738e-02  1.00000000e+00]]
[[ 1.06774679e+00 -5.23448212e+01  2.06926980e+01  5.08764487e+02]
 [-5.05834364e+01 -1.33201976e+01 -1.47406641e+01  4.79228998e+02]
 [ 3.06792659e-03 -3.19008054e-02 -3.43665129e-02  1.00000000e+00]]

```

The top is the estimated camera matrix, and below is the one computed by the creators of the data set. As you can see, they are almost identical. Last, the 3D points are projected using the estimated camera and plotted. The result looks like Figure 5-6 with the true points shown as circles and the estimated camera projection as dots.

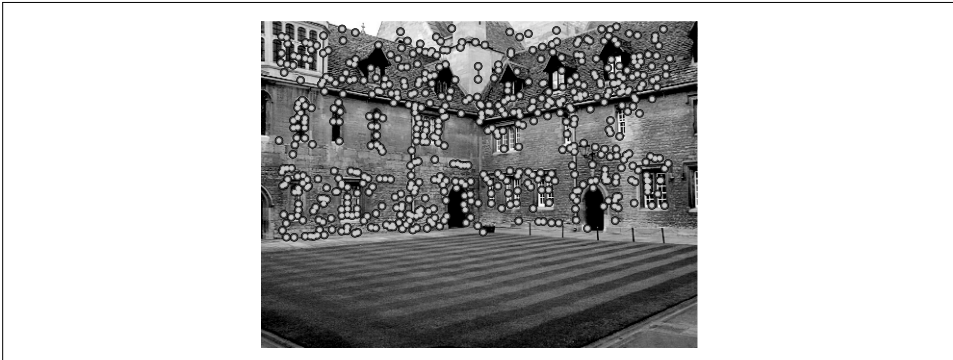


Figure 5-6. Projected points in view 1 computed using an estimated camera matrix.

Computing the Camera Matrix from a Fundamental Matrix

In a two view scenario, the camera matrices can be recovered from the fundamental matrix. Assuming the first camera matrix is normalized to $P_1 = [I \mid 0]$, the problem is to find the second camera matrix P_2 . There are two different cases, the uncalibrated case and the calibrated case.

The uncalibrated case—projective reconstruction

Without any knowledge of the camera's intrinsic parameters, the camera matrix can only be retrieved up to a projective transformation. This means that if the camera pair is used to reconstruct 3D points, the reconstruction is only accurate up to a projective transformation (you can get any solution out of the whole range of projective scene distortions). This means that angles and distances are not respected.

Therefore, in the uncalibrated case the second camera matrix can be chosen up to a (3×3) projective transformation. A simple choice is

$$P_2 = [S_{\mathbf{e}} F \mid \mathbf{e}],$$

where $\mathbf{e}$ is the left epipole, $\mathbf{e}^T F = 0$ and $S_{\mathbf{e}}$ a skew matrix as in equation (5.2). Remember, a triangulation with this matrix will most likely give distortions, for example in the form of skewed reconstructions.

Here is what it looks like in code:

```
def compute_P_from_fundamental(F):  
    """ Computes the second camera matrix (assuming P1 = [I 0])  
    from a fundamental matrix. """  
  
    e = compute_epipole(F.T) # left epipole  
    Te = skew(e)  
    return vstack((dot(Te, F.T).T, e)).T
```

We used the helper function `skew()` defined as:

```
def skew(a):  
    """ Skew matrix A such that a x v = Av for any v. """  
  
    return array([[0, -a[2], a[1]], [a[2], 0, -a[0]], [-a[1], a[0], 0]])
```

Add both these functions to the file `sfm.py`.

The calibrated case—metric reconstruction

With known calibration, the reconstruction will be metric and preserve properties of Euclidean space (except for a global scale parameter). In terms of reconstructing a 3D scene, this calibrated case is the interesting one.

With known calibration K , we can apply its inverse K^{-1} to the image points $\mathbf{x}_K = K^{-1}\mathbf{x}$ so that the camera equation becomes

$$\mathbf{x}_K = K^{-1}K[R \mid \mathbf{t}]\mathbf{X} = [R \mid \mathbf{t}]\mathbf{X},$$

in the new image coordinates. The points in these new image coordinates satisfy the same fundamental equation as before:

$$\mathbf{x}_{K_2}^T F \mathbf{x}_{K_1} = 0.$$

The fundamental matrix for calibration-normalized coordinates is called the *essential matrix* and is usually denoted E instead of F , to make it clear that this is the calibrated case and the image coordinates are normalized.

The camera matrices recovered from an essential matrix respect metric relationships but there are four possible solutions. Only one of them has the scene in front of both cameras, so it is easy to pick the right one.

Here is an algorithm for computing the four solutions (see [13] for the details). Add this function to *sfm.py*:

```
def compute_P_from_essential(E):
    """ Computes the second camera matrix (assuming P1 = [I 0])
        from an essential matrix. Output is a list of four
        possible camera matrices. """

    # make sure E is rank 2
    U,S,V = svd(E)
    if det(dot(U,V))<0:
        V = -V
    E = dot(U,dot(diag([1,1,0]),V))

    # create matrices (Hartley p 258)
    Z = skew([0,0,-1])
    W = array([[0,-1,0],[1,0,0],[0,0,1]])

    # return all four solutions
    P2 = [vstack((dot(U,dot(W,V)).T,U[:,2])).T,
          vstack((dot(U,dot(W,V)).T,-U[:,2])).T,
          vstack((dot(U,dot(W.T,V)).T,U[:,2])).T,
          vstack((dot(U,dot(W.T,V)).T,-U[:,2])).T]

    return P2
```

First, this function makes sure the essential matrix is rank 2 (with two equal non-zero singular values), then the four solutions are created according to the recipe in [13]. A list with four camera matrices is returned. How to pick the right one, we leave to an example later.

This concludes all the theory needed to compute 3D reconstructions from a collection of images.

5.3 Multiple View Reconstruction

Let's look at how to use the concepts above to compute an actual 3D reconstruction from a pair of images. Computing a 3D reconstruction like this is usually referred to as *structure from motion* (*SfM*) since the motion of a camera (or cameras) gives you 3D structure.

Assuming the camera has been calibrated, the steps are as follows:

1. Detect feature points and match them between the two images.
2. Compute the fundamental matrix from the matches.

3. Compute the camera matrices from the fundamental matrix.
4. Triangulate the 3D points.

We have all the tools to do this, but we need a robust way to compute a fundamental matrix when the point correspondences between the images contain incorrect matches.

Robust Fundamental Matrix Estimation

Similar to when we needed a robust way to compute homographies (Section 3.3), we also need to be able to estimate a fundamental matrix when there is noise and incorrect matches. As before, we will use RANSAC, this time combined with the eight point algorithm. It should be mentioned that the eight point algorithm breaks down for planar scenes, so you cannot use it for scenes where the scene points are all on a plane.

Add this class to *sfm.py*:

```
class RansacModel(object):
    """ Class for fundamental matrix fit with ransac.py from
        http://www.scipy.org/Cookbook/RANSAC """

    def __init__(self, debug=False):
        self.debug = debug

    def fit(self, data):
        """ Estimate fundamental matrix using eight
            selected correspondences. """

        # transpose and split data into the two point sets
        data = data.T
        x1 = data[:3,:8]
        x2 = data[3,:8]

        # estimate fundamental matrix and return
        F = compute_fundamental_normalized(x1,x2)
        return F

    def get_error(self, data, F):
        """ Compute  $x^T F x$  for all correspondences,
            return error for each transformed point. """

        # transpose and split data into the two point
        data = data.T
        x1 = data[:3]
        x2 = data[3:]

        # Sampson distance as error measure
        Fx1 = dot(F,x1)
        Fx2 = dot(F,x2)
        denom = Fx1[0]**2 + Fx1[1]**2 + Fx2[0]**2 + Fx2[1]**2
        err = ( diag(dot(x1.T,dot(F,x2))) )**2 / denom

        # return error per point
        return err
```

As before, we need `fit()` and `get_error()` methods. The error measure chosen here is the Sampson distance (see [13]). The `fit()` method now selects eight points and uses a normalized version of the eight point algorithm:

```
def compute_fundamental_normalized(x1,x2):
    """ Computes the fundamental matrix from corresponding points
        (x1,x2 3*n arrays) using the normalized 8 point algorithm. """

    n = x1.shape[1]
    if x2.shape[1] != n:
        raise ValueError("Number of points don't match.")

    # normalize image coordinates
    x1 = x1 / x1[2]
    mean_1 = mean(x1[:2],axis=1)
    S1 = sqrt(2) / std(x1[:2])
    T1 = array([[S1,0,-S1*mean_1[0]], [0,S1,-S1*mean_1[1]], [0,0,1]])
    x1 = dot(T1,x1)

    x2 = x2 / x2[2]
    mean_2 = mean(x2[:2],axis=1)
    S2 = sqrt(2) / std(x2[:2])
    T2 = array([[S2,0,-S2*mean_2[0]], [0,S2,-S2*mean_2[1]], [0,0,1]])
    x2 = dot(T2,x2)

    # compute F with the normalized coordinates
    F = compute_fundamental(x1,x2)

    # reverse normalization
    F = dot(T1.T,dot(F,T2))

    return F/F[2,2]
```

This function normalizes the image points to zero mean and fixed variance.

Now we can use this class in a function. Add the following function to `sfm.py`:

```
def F_from_ransac(x1,x2,model,maxiter=5000,match_threshold=1e-6):
    """ Robust estimation of a fundamental matrix F from point
        correspondences using RANSAC (ransac.py from
        http://www.scipy.org/Cookbook/RANSAC).

        input: x1,x2 (3*n arrays) points in hom. coordinates. """

    import ransac

    data = vstack((x1,x2))

    # compute F and return with inlier index
    F,ransac_data = ransac.ransac(data.T,model,8,maxiter,match_threshold,20,
                                  return_all=True)
    return F, ransac_data['inliers']
```

Here we return the best fundamental matrix F together with the inlier index so that we know what matches were consistent with F . Compared to the homography estimation, we increased the default max iterations and changed the matching threshold, which was in pixels before and is in Sampson distance now.



Figure 5-7. Example image pair of a scene where the images are taken at different viewpoints.

3D Reconstruction Example

In this section, we will see a complete example of reconstructing a 3D scene from start to finish. We will use two images taken with a camera with known calibration. The images are of the famous Alcatraz prison and are shown in Figure 5-7.³

Let's split up the code into a few chunks so that it is easier to follow. First, we extract features, match them, and estimate a fundamental matrix and camera matrices:

```
import homography
import sfm
import sift

# calibration
K = array([[2394,0,932],[0,2398,628],[0,0,1]])

# load images and compute features
im1 = array(Image.open('alcatraz1.jpg'))
sift.process_image('alcatraz1.jpg','im1.sift')
l1,d1 = sift.read_features_from_file('im1.sift')

im2 = array(Image.open('alcatraz2.jpg'))
sift.process_image('alcatraz2.jpg','im2.sift')
l2,d2 = sift.read_features_from_file('im2.sift')

# match features
matches = sift.match_twosided(d1,d2)
ndx = matches.nonzero()[0]

# make homogeneous and normalize with inv(K)
x1 = homography.make_homog(l1[ndx,:2].T)
ndx2 = [int(matches[i]) for i in ndx]
x2 = homography.make_homog(l2[ndx2,:2].T)

x1n = dot(inv(K),x1)
x2n = dot(inv(K),x2)
```

³ Images courtesy of Carl Olsson (<http://www.maths.lth.se/matematiklth/personal/callef/>).

```

# estimate E with RANSAC
model = sfm.RansacModel()
E,inliers = sfm.F_from_ransac(x1n,x2n,model)

# compute camera matrices (P2 will be list of four solutions)
P1 = array([[1,0,0,0],[0,1,0,0],[0,0,1,0]])
P2 = sfm.compute_P_from_essential(E)

```

The calibration is known, so here we just hardcode the K matrix at the beginning. As in earlier examples, we pick out the points that belong to matches. After that, we normalize them with K^{-1} and run the RANSAC estimation with the normalized eight point algorithm. Since the points are normalized, this gives us an essential matrix. We make sure to keep the index of the inliers, as we will need them. From the essential matrix we compute the four possible solutions of the second camera matrix.

From the list of camera matrices, we pick the one that has the most scene points in front of both cameras after triangulation:

```

# pick the solution with points in front of cameras
ind = 0
maxres = 0
for i in range(4):
    # triangulate inliers and compute depth for each camera
    X = sfm.triangulate(x1n[:,inliers],x2n[:,inliers],P1,P2[i])
    d1 = dot(P1,X)[2]
    d2 = dot(P2[i],X)[2]
    if sum(d1>0)+sum(d2>0) > maxres:
        maxres = sum(d1>0)+sum(d2>0)
        ind = i
        infront = (d1>0) & (d2>0)

# triangulate inliers and remove points not in front of both cameras
X = sfm.triangulate(x1n[:,inliers],x2n[:,inliers],P1,P2[ind])
X = X[:,infront]

```

We loop through the four solutions and each time triangulate the 3D points corresponding to the inliers. The sign of the depth is given by the third value of each image point after projecting the triangulated X back to the images. We keep the index with the most positive depths and also store a boolean for each point in the best solution so that we can pick only the ones that actually are in front. Due to noise and errors in all of the estimations done, there is a risk that some points still are behind one camera, even with the correct camera matrices. Once we have the right solution, we triangulate the inliers and keep the points in front of the cameras.

Now we can plot the reconstruction:

```

# 3D plot
from mpl_toolkits.mplot3d import axes3d

fig = figure()
ax = fig.gca(projection='3d')
ax.plot(-X[0],X[1],X[2],'k.')
axis('off')

```

The 3D plots with `mplot3d` have the first axis reversed compared to our coordinate system, so we change the sign.

We can then plot the reprojection in each view:

```
# plot the projection of X
import camera

# project 3D points
cam1 = camera.Camera(P1)
cam2 = camera.Camera(P2[ind])
x1p = cam1.project(X)
x2p = cam2.project(X)

# reverse K normalization
x1p = dot(K,x1p)
x2p = dot(K,x2p)

figure()
imshow(im1)
gray()
plot(x1p[0],x1p[1],'o')
plot(x1[0],x1[1],'r.')
axis('off')

figure()
imshow(im2)
gray()
plot(x2p[0],x2p[1],'o')
plot(x2[0],x2[1],'r.')
axis('off')
show()
```

After projecting the 3D points, we need to reverse the initial normalization by multiplying with the calibration matrix.

The result looks like Figure 5-8. As you can see, the reprojected points don't exactly match the original feature locations, but they are reasonably close. It is possible to further refine the camera matrices to improve the reconstruction and reprojection, but that is outside the scope of this simple example.

Extensions and More Than Two Views

There are some steps and further extensions to multiple view reconstructions that we cannot cover in a book like this. Here are some of them with references for further reading.

More views

With more than two views of the same scene, the 3D reconstruction will usually be more accurate and more detailed. Since the fundamental matrix only relates a pair of views, the process is a little different with many images.

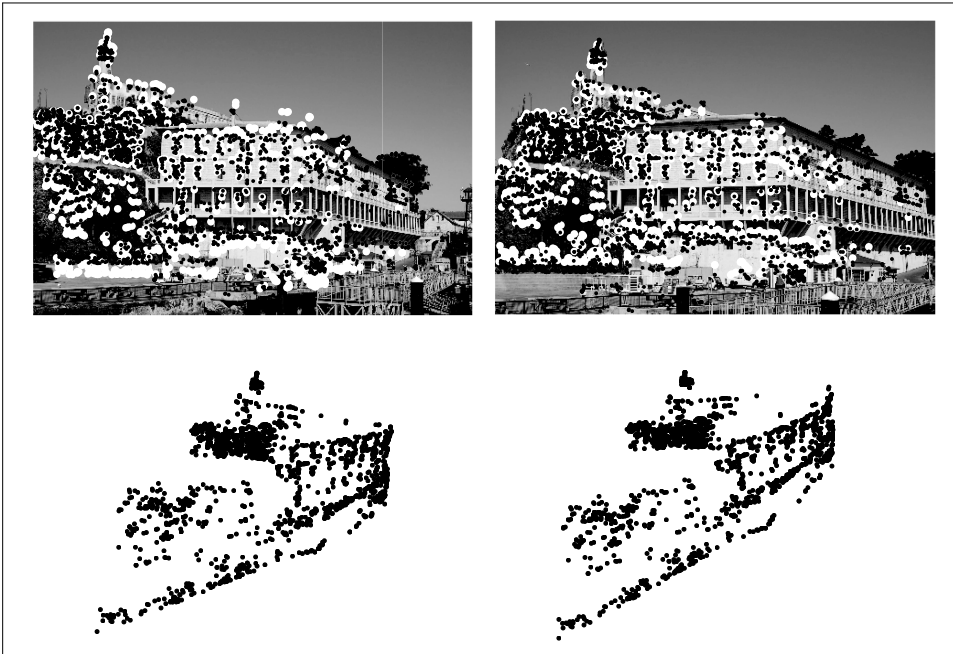


Figure 5-8. Example of computing a 3D reconstruction from a pair of images using image matches: the two images with feature points shown in black and reprojected reconstructed 3D points shown in white (top); the 3D reconstruction (bottom).

For video sequences, we can use the temporal aspect and match features in consecutive frame pairs. The relative orientation needs to be added incrementally from each pair to the next (similar to how we added homographies in the panorama example in Figure 3-12). This approach usually works well, and tracking can be used to effectively find correspondences (see Section 10.4 for more on tracking). One problem is that errors will accumulate the more views that are added. This can be fixed with a final optimization step; see below.

With still images, one approach is to find a central reference view and compute all the other camera matrices relative to that one. Another method is to compute camera matrices and a 3D reconstruction for one image pair and then incrementally add new images and 3D points; see for example [34]. As a side note, there are ways to compute 3D and camera positions from three views at the same time (see for example [13]), but beyond that an incremental approach is needed.

Bundle adjustment

From our simple 3D reconstruction example in Figure 5-8, it is clear that there will be errors in the position of the recovered points and in the camera matrices computed from the estimated fundamental matrix. With more views, the errors will accumulate. Therefore, a final step in multiple view reconstructions is often to try to minimize the reprojection errors by optimizing the position of the 3D points and the camera

parameters. This process is called *bundle adjustment*. Details can be found in [13] and [35] and a short overview at http://en.wikipedia.org/wiki/Bundle_adjustment.

Self-calibration

In the case of uncalibrated cameras, it is sometimes possible to compute the calibration from image features. This process is called *self-calibration*. There are many different algorithms, depending on what assumptions can be made on parameters of the camera calibration matrix and depending on what types of image data is available (feature matches, parallel lines, planes, etc.). The interested reader can take a look at [13] and [26, Chap. 6].

As a side note to calibration, there is a useful script, *extract_focal.pl*, as part of the Bundler SfM system (<http://phototour.cs.washington.edu/bundler/>). This uses a lookup table for common cameras and estimates the focal length based on the image EXIF data.

5.4 Stereo Images

A special case of multi-view imaging is *stereo vision* (or *stereo imaging*), where two cameras are observing the same scene with only a horizontal (sideways) displacement between the cameras. When the cameras are configured so that the two images have the same image plane with the image rows vertically aligned, the image pair is said to be *rectified*. This is common in robotics, and such a setup is often called a *stereo rig*.

Any stereo camera setup can be rectified by warping the images to a common plane so that the epipolar lines are image rows (a stereo rig is usually constructed to give such rectified image pairs). This is outside the scope of this section, but the interested reader can find the details in [13, p. 303] or [3, p. 430].

Assuming that the two images are rectified, finding correspondences is constrained to searching along image rows. Once a corresponding point is found, its depth (Z coordinate) can be computed directly from the horizontal displacement as it is inversely proportional to the displacement,

$$Z = \frac{fb}{x_l - x_r},$$

where f is the rectified image focal length, b the distance between the camera centers, and x_l and x_r the x-coordinate of the corresponding point in the left and right image. The distance separating the camera centers is called the *baseline*. Figure 5-9 illustrates a rectified stereo camera setup.

Stereo reconstruction (sometimes called *dense depth reconstruction*) is the problem of recovering a depth map (or, inversely, a disparity map) where the depth (or disparity) for each pixel in the image is estimated. This is a classic problem in computer vision and there are many algorithms for solving it. The Middlebury Stereo Vision Page (<http://vision.middlebury.edu/stereo/>) contains a constantly updated evaluation of the best algorithms with code and descriptions of many implementations. In the next

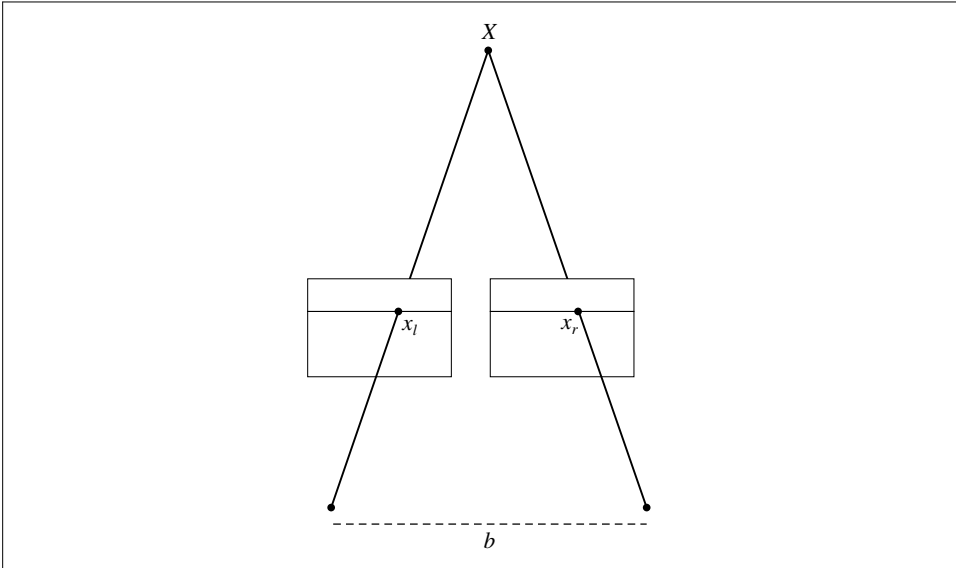


Figure 5-9. An illustration of a rectified stereo image setup where corresponding points are on the same row in both images.

section, we will implement a stereo reconstruction algorithm based on normalized cross-correlation.

Computing Disparity Maps

In this stereo reconstruction algorithm, we will try a range of displacements and record the best displacement for each pixel by selecting the one with the best score according to normalized cross-correlation of the local image neighborhood. This is sometimes called plane sweeping, since each displacement step corresponds to a plane at some depth. While not exactly state of the art in stereo reconstruction, this is a simple method that usually gives decent results.

Normalized cross-correlation can be efficiently computed when applied densely across images. This is different from when we applied it between sparse point correspondences in Chapter 2. We want to evaluate normalized cross-correlation on a patch (basically a local neighborhood) around each pixel. For this case, we can rewrite the NCC around a pixel, equation (2.3), as

$$\text{ncc}(I_1, I_2) = \frac{\sum_{\mathbf{x}} (I_1(\mathbf{x}) - \mu_1)(I_2(\mathbf{x}) - \mu_2)}{\sqrt{\sum_{\mathbf{x}} (I_1(\mathbf{x}) - \mu_1)^2 \sum_{\mathbf{x}} (I_2(\mathbf{x}) - \mu_2)^2}},$$

where we skip the normalizing constant in front (it is not needed here) and the sums are taken over the pixels of a local patch around the pixel.

Now, we want this for every pixel in the image. The three sums are over a local patch region and can be computed efficiently using image filters, just as we did for blur

and derivatives. The function `uniform_filter()` in the `ndimage.filters` module will compute the sums over a rectangular patch.

Here's the function that does the plane sweep and returns the best disparity for each pixel. Create a file *stereo.py* and add the following:

```
def plane_sweep_ncc(im_l, im_r, start, steps, wid):
    """ Find disparity image using normalized cross-correlation. """

    m, n = im_l.shape

    # arrays to hold the different sums
    mean_l = zeros((m, n))
    mean_r = zeros((m, n))
    s = zeros((m, n))
    s_l = zeros((m, n))
    s_r = zeros((m, n))

    # array to hold depth planes
    dmaps = zeros((m, n, steps))

    # compute mean of patch
    filters.uniform_filter(im_l, wid, mean_l)
    filters.uniform_filter(im_r, wid, mean_r)

    # normalized images
    norm_l = im_l - mean_l
    norm_r = im_r - mean_r

    # try different disparities
    for displ in range(steps):
        # move left image to the right, compute sums
        filters.uniform_filter(roll(norm_l, -displ-start)*norm_r, wid, s) # sum nominator
        filters.uniform_filter(roll(norm_l, -displ-start)*roll(norm_l, -displ-start), wid,
                               s_l)
        filters.uniform_filter(norm_r*norm_r, wid, s_r) # sum denominator

        # store ncc scores
        dmaps[:, :, displ] = s/sqrt(s_l*s_r)

    # pick best depth for each pixel
    return argmax(dmaps, axis=2)
```

First, we need to create some arrays to hold the filtering results as `uniform_filter()` takes them as input arguments. Then, we create an array to hold each of the planes so that we can apply `argmax()` along the last dimension to find the best depth for each pixel. The function iterates over all *steps* displacements from *start*. One image is shifted using the `roll()` function, and the three sums of the NCC are computed using filtering.

Here is a full example of loading images and computing the displacement map using this function:

```

import stereo

im_l = array(Image.open('scene1.row3.col3.ppm').convert('L'),'f')
im_r = array(Image.open('scene1.row3.col4.ppm').convert('L'),'f')

# starting displacement and steps
steps = 12
start = 4

# width for ncc
wid = 9

res = stereo.plane_sweep_ncc(im_l,im_r,start,steps,wid)

import scipy.misc
scipy.misc.imsave('depth.png',res)

```

Here we first load a pair of images from the classic “tsukuba” set and convert them to grayscale. Next, we set the parameters needed for the plane sweep function, the number of displacements to try, the starting value and the width of the NCC patch. You will notice that this method is fairly fast, at least compared to matching features with NCC. This is because everything is computed using filters.

This approach also works for other filters. The uniform filter gives all pixels in a square patch equal weight, but in some cases other filters for the NCC computation might be preferred. Here is one alternative using a Gaussian filter that produces smoother disparity maps. Add this to *stereo.py*:

```

def plane_sweep_gauss(im_l,im_r,start,steps,wid):
    """ Find disparity image using normalized cross-correlation
        with Gaussian weighted neighborhoods. """

    m,n = im_l.shape

    # arrays to hold the different sums
    mean_l = zeros((m,n))
    mean_r = zeros((m,n))
    s = zeros((m,n))
    s_l = zeros((m,n))
    s_r = zeros((m,n))

    # array to hold depth planes
    dmaps = zeros((m,n,steps))

    # compute mean
    filters.gaussian_filter(im_l,wid,0,mean_l)
    filters.gaussian_filter(im_r,wid,0,mean_r)

    # normalized images
    norm_l = im_l - mean_l
    norm_r = im_r - mean_r

```

```

# try different disparities
for displ in range(steps):
    # move left image to the right, compute sums
    filters.gaussian_filter(roll(norm_l,-displ-start)*norm_r,wid,0,s) # sum nominator
    filters.gaussian_filter(roll(norm_l,-displ-start)*roll(norm_l,-displ-start),wid,
                           0,s_l)
    filters.gaussian_filter(norm_r*norm_r,wid,0,s_r) # sum denominator

    # store ncc scores
    dmaps[:,displ] = s/sqrt(s_l*s_r)

# pick best depth for each pixel
return argmax(dmaps,axis=2)

```

The code is the same as for the uniform filter with the exception of the extra argument in the filtering. We need to pass a zero to `gaussian_filter()` to indicate that we want a standard Gaussian and not any derivatives (see page 18 for details).

Use this function the same way as the previous plane sweep function. Figures 5-10 and 5-11 show some results of these two plane sweep implementations on some standard stereo benchmark images. The images are from [29] and [30] and are available at <http://vision.middlebury.edu/stereo/data/>. Here we used the “tsukuba” and “cones” images and set *wid* to 9 in the standard version and 3 for the Gaussian version. The top row shows the image pair, bottom left is the standard NCC plane sweep, and bottom right

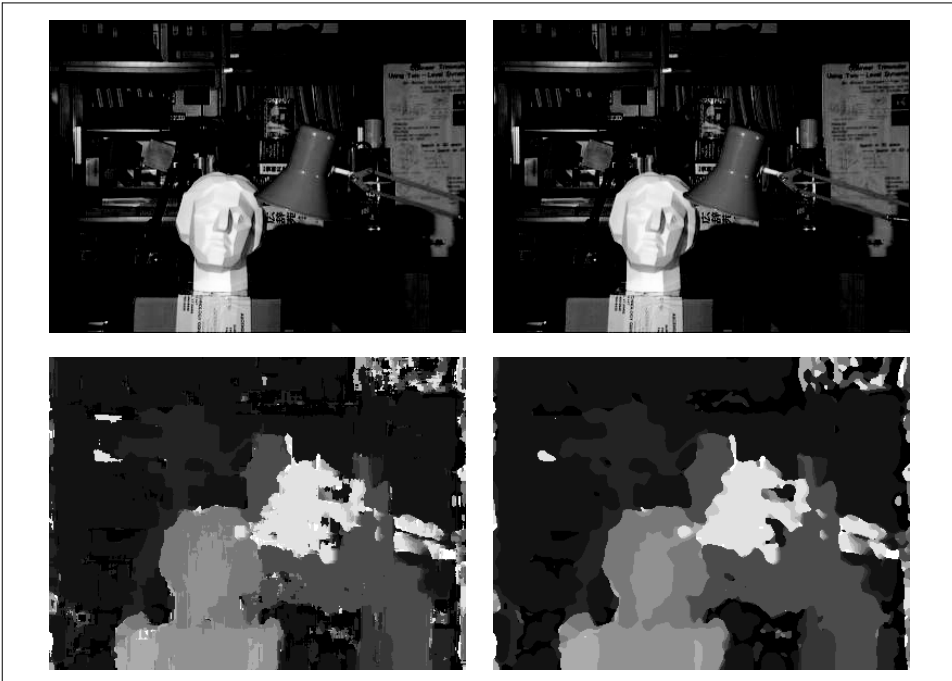


Figure 5-10. Example of computing disparity maps from a stereo image pair with normalized cross-correlation.

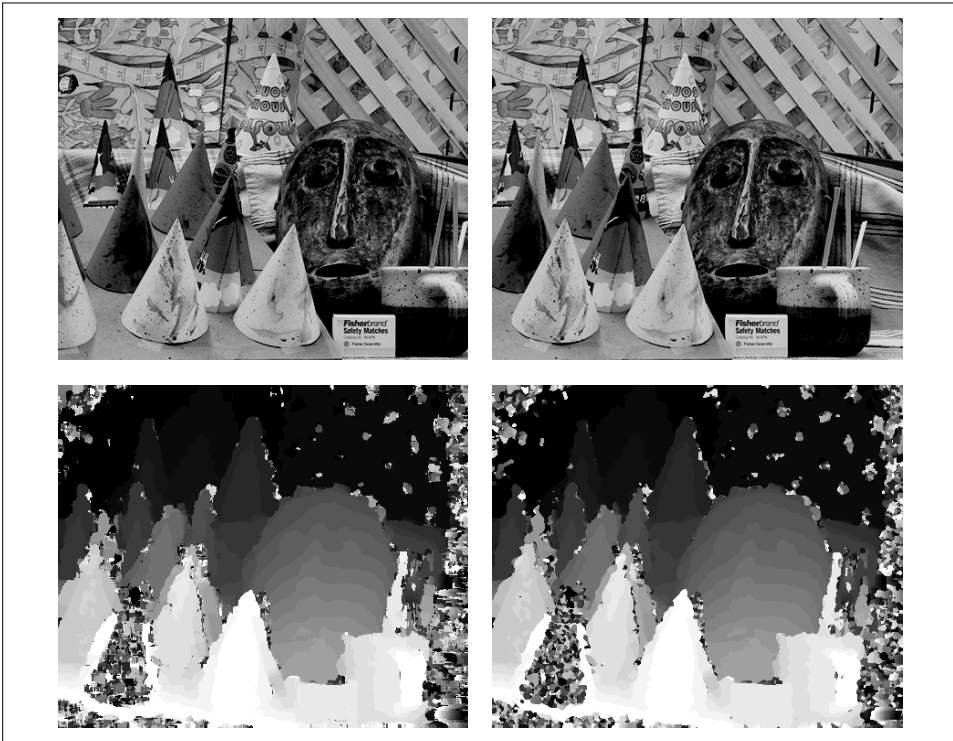


Figure 5-11. Example of computing disparity maps from a stereo image pair with normalized cross-correlation.

right is the Gaussian version. As you can see, the Gaussian version is less noisy but also has less detail than the standard version.

Exercises

1. Use the techniques introduced in this chapter to verify matches in the White House example on page 46 (or even better, an example of your own) and see if you can improve on the results.
2. Compute feature matches for an image pair and estimate the fundamental matrix. Use the epipolar lines to do a second pass to find more matches by searching for the best match along the epipolar line for each feature.
3. Take a set with three or more images. Pick one pair and compute 3D points and camera matrices. Match features to the remaining images to get correspondences. Then take the 3D points for the correspondences and compute camera matrices for the other images using resection. Plot the 3D points and the camera positions. Use a set of your own or one of the Oxford multi-view sets.
4. Implement a stereo version that uses sum of squared differences (SSD) instead of NCC using filtering the same way as in the NCC example.

5. Try smoothing the stereo depth maps using the ROF de-noising from Section 1.5. Experiment with the size of the cross-correlation patches to get sharp edges with noise levels that can be removed with smoothing.
6. One way to improve the quality of the disparity maps is to compare the disparities from moving the left image to the right and the right image to the left, and only keep the parts that are consistent. This will, for example, clean up the parts where there is occlusion. Implement this idea and compare the results to the one-directional plane sweeping.
7. The New York Public Library has many old historic stereo photographs. Browse the gallery at <http://stereo.nypl.org/gallery> and download some images you like (you can right click and save JPEGs). The images should be rectified already. Cut the image in two parts and try the dense depth reconstruction code.

Clustering Images

This chapter introduces several clustering methods and shows how to use them for clustering images for finding groups of similar images. Clustering can be used for recognition, for dividing data sets of images, and for organization and navigation. We also look at using clustering for visualizing similarity between images.

6.1 *K*-Means Clustering

K-means is a very simple clustering algorithm that tries to partition the input data in k clusters. *K-means* works by iteratively refining an initial estimate of *class centroids* as follows:

1. Initialize centroids μ_i , $i = 1 \dots k$, randomly or with some guess.
2. Assign each data point to the class c_i of its nearest centroid.
3. Update the centroids as the average of all data points assigned to that class.
4. Repeat 2 and 3 until convergence.

K-means tries to minimize the *total within-class variance*

$$V = \sum_{i=1}^k \sum_{\mathbf{x}_j \in c_i} (\mathbf{x}_j - \mu_i)^2,$$

where $\mathbf{x}_j$ are the data vectors. The algorithm above is a heuristic refinement algorithm that works fine for most cases, but it does not guarantee that the best solution is found. To avoid the effects of choosing a bad centroid initialization, the algorithm is often run several times with different initialization centroids. Then the solution with lowest variance V is selected.

The main drawback of this algorithm is that the number of clusters needs to be decided beforehand, and an inappropriate choice will give poor clustering results. The benefits are that it is simple to implement, it is parallelizable, and it works well for a large range of problems without any need for tuning.

The SciPy Clustering Package

Although simple to implement, there is no need to. The SciPy vector quantization package `scipy.cluster.vq` comes with a k -means implementation. Here's how to use it.

Let's start with creating some sample 2D data to illustrate:

```
from scipy.cluster.vq import *

class1 = 1.5 * randn(100,2)
class2 = randn(100,2) + array([5,5])
features = vstack((class1,class2))
```

This generates two normally distributed classes in two dimensions. To try to cluster the points, run k -means with $k = 2$ like this:

```
centroids,variance = kmeans(features,2)
```

The variance is returned but we don't really need it, since the SciPy implementation computes several runs (default is 20) and selects the one with smallest variance for us. Now you can check where each data point is assigned using the vector quantization function in the SciPy package:

```
code,distance = vq(features,centroids)
```

By checking the value of `code`, we can see if there are any incorrect assignments. To visualize, we can plot the points and the final centroids:

```
figure()
ndx = where(code==0)[0]
plot(features[ndx,0],features[ndx,1], '*')
ndx = where(code==1)[0]
plot(features[ndx,0],features[ndx,1], 'r.')
plot(centroids[:,0],centroids[:,1], 'go')
axis('off')
show()
```

Here the function `where()` gives the indices for each class. This should give a plot like the one in Figure 6-1.

Clustering Images

Let's try k -means on the font images described on page 14. The file *selectedfontimages.zip* contains 66 images from this font data set (these are selected for easy overview when illustrating the clusters). As descriptor vector for each image, we will use the projection coefficients after projecting on the 40 first principal components computed earlier. Loading the model file using `pickle`, projecting the images on the principal components, and clustering is then done like this:

```
import imtools
import pickle
from scipy.cluster.vq import *

# get list of images
imlist = imtools.get_imlist('selected_fontimages/')
imnbr = len(imlist)
```

```

# load model file
with open('a_pca_modes.pkl','rb') as f:
    immean = pickle.load(f)
    V = pickle.load(f)

# create matrix to store all flattened images
immatrix = array([array(Image.open(im)).flatten()
                  for im in imlist], 'f')

# project on the 40 first PCs
immean = immean.flatten()
projected = array([dot(V[:40], immatrix[i]-immean) for i in range(imnbr)])

# k-means
projected = whiten(projected)
centroids, distortion = kmeans(projected, 4)

code, distance = vq(projected, centroids)

```

Same as before, *code* contains the cluster assignment for each image. In this case, we tried $k = 4$. We also chose to “whiten” the data using SciPy’s `whiten()`, normalizing so that each feature has unit variance. Try to vary parameters like the number of principal components used and the value of k to see how the clustering results change. The clusters can be visualized like this:

```

# plot clusters
for k in range(4):
    ind = where(code==k)[0]
    figure()
    gray()
    for i in range(minimum(len(ind), 40)):
        subplot(4, 10, i+1)
        imshow(immatrix[ind[i]].reshape((25, 25)))
        axis('off')
show()

```

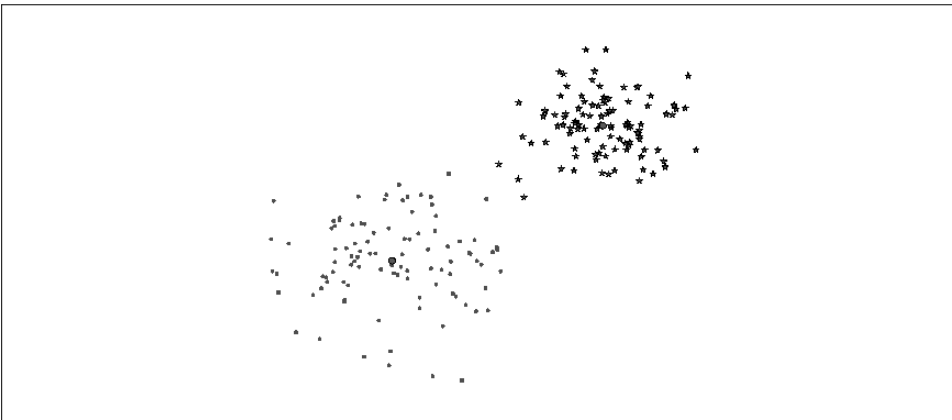


Figure 6-1. An example of k -means clustering of 2D points. Class centroids are marked as large rings and the predicted classes are stars and dots, respectively.

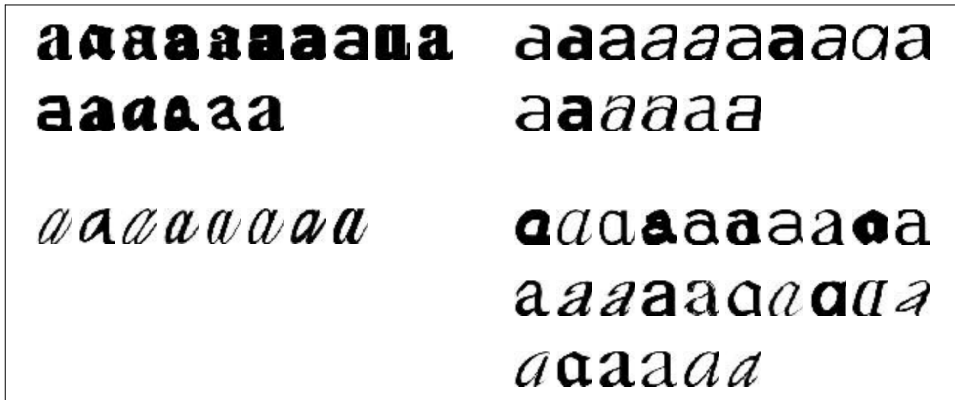


Figure 6-2. An example of k -means clustering with $k = 4$ of the font images using 40 principal components.

Here we show each cluster in a separate figure window in a grid with maximum 40 images from the cluster shown. We use the PyLab function `subplot()` to define the grid. A sample cluster result can look like the one in Figure 6-2.

For more details on the k -means SciPy implementation and the `scipy.cluster.vq` package, see the reference guide <http://docs.scipy.org/doc/scipy/reference/cluster.vq.html>.

Visualizing the Images on Principal Components

To see how the clustering using just a few principal components as above can work, we can visualize the images on their coordinates in a pair of principal component directions. One way is to project on two components by changing the projection to

```
projected = array([dot(V[[0,2]],immatrix[i]-immean) for i in range(imnbr)])
```

to get only the relevant coordinates (in this case $V[[0, 2]]$ gives the first and third). Alternatively, project on all components and afterward just pick out the columns you need.

For the visualization, we will use the `ImageDraw` module in PIL. Assuming that you have the projected images and image list as above, the following short script will generate a plot like the one in Figure 6-3:

```
from PIL import Image, ImageDraw

# height and width
h,w = 1200,1200

# create a new image with a white background
img = Image.new('RGB', (w,h), (255,255,255))
draw = ImageDraw.Draw(img)

# draw axis
draw.line((0,h/2,w,h/2),fill=(255,0,0))
draw.line((w/2,0,w/2,h),fill=(255,0,0))
```

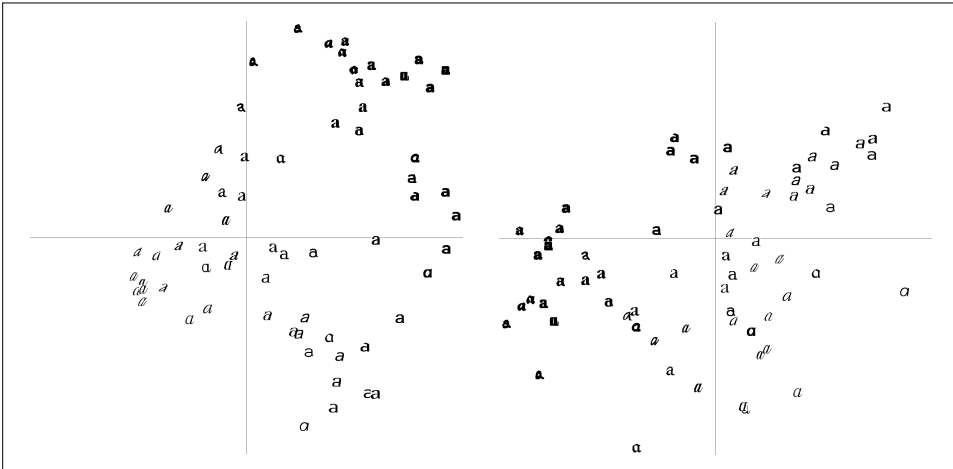


Figure 6-3. The projection of the font images on pairs of principal components: the first and second principal components (left); the second and third (right).

```
# scale coordinates to fit
scale = abs(projected).max(0)
scaled = floor(array([(p / scale) * (w/2-20,h/2-20) +
                    (w/2,h/2) for p in projected]))

# paste thumbnail of each image
for i in range(imnbr):
    nodeim = Image.open(imlist[i])
    nodeim.thumbnail((25,25))
    ns = nodeim.size
    img.paste(nodeim,(scaled[i][0]-ns[0]//2,scaled[i][1]-
                    ns[1]//2,scaled[i][0]+ns[0]//2+1,scaled[i][1]+ns[1]//2+1))

img.save('pca_font.jpg')
```

Here we used the integer or floor division operator `//`, which returns an integer pixel position by removing any values after the decimal point.

Plots like these illustrate how the images are distributed in the 40 dimensions and can be very useful for choosing a good descriptor. Already in just these two-dimensional projections the closeness of similar font images is clearly visible.

Clustering Pixels

Before closing this section, we will take a look at an example of clustering individual pixels instead of entire images. Grouping image regions and pixels into “meaningful” components is called *image segmentation* and will be the topic of Chapter 9. Naively applying *k*-means on the pixel values will not give anything meaningful except in very simple images. More sophisticated class models than average pixel color or spatial consistency is needed to produce useful results. For now, let’s just apply *k*-means to

the RGB values and worry about solving segmentation problems later (Section 9.2 has the details).

The following code sample takes an image, reduces it to a lower resolution version with pixels as mean values of the original image regions (taken over a square grid of size $steps \times steps$), and clusters the regions using k -means:

```
from scipy.cluster.vq import *
from scipy.misc import imread

steps = 50 # image is divided in steps*steps region
im = array(Image.open('empire.jpg'))

dx = im.shape[0] / steps
dy = im.shape[1] / steps

# compute color features for each region
features = []
for x in range(steps):
    for y in range(steps):
        R = mean(im[x*dx:(x+1)*dx, y*dy:(y+1)*dy, 0])
        G = mean(im[x*dx:(x+1)*dx, y*dy:(y+1)*dy, 1])
        B = mean(im[x*dx:(x+1)*dx, y*dy:(y+1)*dy, 2])
        features.append([R,G,B])
features = array(features, 'f') # make into array

# cluster
centroids, variance = kmeans(features, 3)
code, distance = vq(features, centroids)

# create image with cluster labels
codeim = code.reshape(steps, steps)
codeim = imread(codeim, im.shape[:2], interp='nearest')

figure()
imshow(codeim)
show()
```

The input to k -means is an array with $steps \times steps$ rows, each containing the R, G, and B mean values. To visualize the result, we use SciPy's `imread()` function to show the $steps \times steps$ image at the original image coordinates. The parameter *interp* specifies what type of interpolation to use; here we use nearest neighbor so we don't introduce new pixel values at the transitions between classes.

Figure 6-4 shows results using 50×50 and 100×100 regions for two relatively simple example images. Note that the ordering of the k -means labels (in this case the colors in the result images) is arbitrary. As you can see, the result is noisy despite down-sampling to only use a few regions. There is no spatial consistency and it is hard to separate regions, like the boy and the grass in the lower example. Spatial consistency and better separation will be dealt with later, together with other image segmentation algorithms. Now let's move on to the next basic clustering algorithm.



Figure 6-4. Clustering of pixels based on their color value using k -means: original image (left); cluster result with $k = 3$ and 50×50 resolution (middle); cluster result with $k = 3$ and 100×100 resolution (right).

6.2 Hierarchical Clustering

Hierarchical clustering (or *agglomerative clustering*) is another simple but powerful clustering algorithm. The idea is to build a similarity tree based on pairwise distances. The algorithm starts with grouping the two closest objects (based on the distance between feature vectors) and creates an “average” node in a tree with the two objects as children. Then the next closest pair is found among the remaining objects but then also including any average nodes, and so on. At each node, the distance between the two children is also stored. Clusters can then be extracted by traversing this tree and stopping at nodes with distance smaller than some threshold that then determines the cluster size.

Hierarchical clustering has several benefits. For example, the tree structure can be used to visualize relationships and show how clusters are related. A good feature vector will give a nice separation in the tree. Another benefit is that the tree can be reused with

different cluster thresholds without having to recompute the tree. The drawback is that one needs to choose a threshold if the actual clusters are needed.

Let's see what this looks like in code.¹ Create a file *hcluster.py* and add the following code (inspired by the hierarchical clustering example in [31]):

```
from itertools import combinations

class ClusterNode(object):
    def __init__(self, vec, left, right, distance=0.0, count=1):
        self.left = left
        self.right = right
        self.vec = vec
        self.distance = distance
        self.count = count # only used for weighted average

    def extract_clusters(self, dist):
        """ Extract list of sub-tree clusters from
            hcluster tree with distance<dist. """
        if self.distance < dist:
            return [self]
        return self.left.extract_clusters(dist) + self.right.extract_clusters(dist)

    def get_cluster_elements(self):
        """ Return ids for elements in a cluster sub-tree. """
        return self.left.get_cluster_elements() + self.right.get_cluster_elements()

    def get_height(self):
        """ Return the height of a node,
            height is sum of each branch. """
        return self.left.get_height() + self.right.get_height()

    def get_depth(self):
        """ Return the depth of a node, depth is
            max of each child plus own distance. """
        return max(self.left.get_depth(), self.right.get_depth()) + self.distance

class ClusterLeafNode(object):
    def __init__(self, vec, id):
        self.vec = vec
        self.id = id

    def extract_clusters(self, dist):
        return [self]

    def get_cluster_elements(self):
        return [self.id]

    def get_height(self):
        return 1
```

¹ There is also a version of hierarchical clustering in the SciPy clustering package that you can look at if you like. We will not use that version here since we want a class that can draw dendrograms and visualize clusters using image thumbnails.


```

def get_depth(self):
    return 0

def L2dist(v1,v2):
    return sqrt(sum((v1-v2)**2))

def L1dist(v1,v2):
    return sum(abs(v1-v2))

def hcluster(features,distfcn=L2dist):
    """ Cluster the rows of features using
        hierarchical clustering. """

    # cache of distance calculations
    distances = {}

    # initialize with each row as a cluster
    node = [ClusterLeafNode(array(f),id=i) for i,f in enumerate(features)]

    while len(node)>1:
        closest = float('Inf')

        # loop through every pair looking for the smallest distance
        for ni,nj in combinations(node,2):
            if (ni,nj) not in distances:
                distances[ni,nj] = distfcn(ni.vec,nj.vec)

            d = distances[ni,nj]
            if d<closest:
                closest = d
                lowestpair = (ni,nj)
        ni,nj = lowestpair

        # average the two clusters
        new_vec = (ni.vec + nj.vec) / 2.0

        # create new node
        new_node = ClusterNode(new_vec,left=ni,right=nj,distance=closest)
        node.remove(ni)
        node.remove(nj)
        node.append(new_node)

    return node[0]

```

We created two classes for tree nodes, `ClusterNode` and `ClusterLeafNode`, to be used to create the cluster tree. The function `hcluster()` builds the tree. First, a list of leaf nodes is created, then the closest pairs are iteratively grouped together based on the distance measure chosen. Returning the final node will give you the root of the tree. Running `hcluster()` on a matrix with feature vectors as rows will create and return the cluster tree.

The choice of distance measure depends on the actual feature vectors. Here, we used the Euclidean (L_2) distance (a function for L_1 distance is also provided), but you can

create any function and use that as parameter to `hcluster()`. We also used the average feature vector of all nodes in a sub-tree as a new feature vector to represent the sub-tree and treat each sub-tree as objects. There are other choices for deciding which two nodes to merge next, such as using *single linking* (use the minimum distance between objects in two sub-trees) and *complete linking* (use the maximum distance between objects in two sub-trees). The choice of linking will affect the type of clusters produced.

To extract the clusters from the tree, you need to traverse the tree from the top until a node with distance value smaller than some threshold is found. This is easiest done recursively. The `ClusterNode` method `extract_clusters()` handles this by returning a list with the node itself if below the distance threshold, and otherwise calling the child nodes (leaf nodes always return themselves). Calling this function will return a list of sub-trees containing the clusters. To get the leaf nodes for each cluster sub-tree that contains the object ids, traverse each sub-tree and return a list of leaves using the method `get_cluster_elements()`.

Let's try this on a simple example to see it all in action. First create some 2D data points (same as for *k*-means above):

```
class1 = 1.5 * randn(100,2)
class2 = randn(100,2) + array([5,5])
features = vstack((class1,class2))
```

Cluster the points and extract the clusters from the list using some threshold (here we used 5) and print the clusters in the console:

```
import hcluster

tree = hcluster.hcluster(features)

clusters = tree.extract_clusters(5)

print 'number of clusters', len(clusters)
for c in clusters:
    print c.get_cluster_elements()
```

This should give a printout similar to this:

```
number of clusters 2

[184, 187, 196, 137, 174, 102, 147, 145, 185, 109, 166, 152, 173, 180, 128, 163, 141,
178, 151, 158, 108, 182, 112, 199, 100, 119, 132, 195, 105, 159, 140, 171, 191, 164,
130, 149, 150, 157, 176, 135, 123, 131, 118, 170, 143, 125, 127, 139, 179, 126, 160,
162, 114, 122, 103, 146, 115, 120, 142, 111, 154, 116, 129, 136, 144, 167, 106, 107,
198, 186, 153, 156, 134, 101, 110, 133, 189, 168, 183, 148, 165, 172, 188, 138, 192,
104, 124, 113, 194, 190, 161, 175, 121, 197, 177, 193, 169, 117, 155]

[56, 4, 47, 18, 51, 95, 29, 91, 23, 80, 83, 3, 54, 68, 69, 5, 21, 1, 44, 57, 17, 90,
30, 22, 63, 41, 7, 14, 59, 96, 20, 26, 71, 88, 86, 40, 27, 38, 50, 55, 67, 8, 28, 79,
64, 66, 94, 33, 53, 70, 31, 81, 9, 75, 15, 32, 89, 6, 11, 48, 58, 2, 39, 61, 45,
65, 82, 93, 97, 52, 62, 16, 43, 84, 24, 19, 74, 36, 37, 60, 87, 92, 181, 99, 10, 49,
12, 76, 98, 46, 72, 34, 35, 13, 73, 78, 25, 42, 77, 85]
```

Ideally, you should get two clusters, but depending on the actual data you might get three or even more. In this simple example of clustering 2D points, one cluster should contain values lower than 100 and the other values 100 and above.